

ABSTRACT

Crop yield prediction is crucial for modern agriculture, aiding farmers, policymakers, and stakeholders in making data-driven decisions to optimize farm management, mitigate risks. The integration of machine learning, satellite data, and remote sensing technologies has revolutionized crop yield prediction, making it a key component of precision agriculture. Machine learning algorithms such as Linear Regression, Support Vector Regression, Random Forest, and Gradient Boosting are used for Crop Yield Prediction (CYP) due to their ability to model complex, nonlinear relationships between various agricultural and environmental factors, enabling accurate and reliable yield estimation. In this paper, the development of a random forest-based crop yield prediction model is proposed as it effectively handles complex, and non-linear relationships. The model provides accurate crop yield predictions, enabling farmers to make informed decisions regarding crop selection, resource allocation, and management practices. The use of Random Forest ensures robustness and generalizability, making it adaptable to different regions with varying climatic and agricultural conditions. The benefits of the model include improved crop planning, optimized resource management, risk mitigation, and scalability across diverse agricultural landscapes. By integrating the model into a web application, it offers a user-friendly platform for stakeholders to interact with and access predictions.

The major advantage of this satellite-driven approach is its scalability and adaptability to different geographic regions and climatic conditions. Since satellite data is globally available and updated frequently, the system can be expanded to monitor multiple crops across various districts or states. It also reduces reliance on manual data collection, enabling near real-time monitoring and forecasting. This project contributes to the growing field of digital agriculture by offering a scalable, accurate, and accessible solution for regional crop yield estimation, promoting sustainable and efficient agricultural practices.

CHAPTER 1 - INTRODUCTION

1.1 ABOUT THE PROJECT

The project titled **“Satellite-Driven Machine Learning Models for Regional Crop Yield Estimation”** is an advanced application of artificial intelligence and remote sensing technologies aimed at transforming the agricultural sector. Accurate estimation of crop yield at a regional level plays a vital role in ensuring food security, optimizing supply chain operations, and supporting agricultural policies. Traditional methods of yield estimation often involve manual surveys and field inspections, which are time-consuming, expensive, and prone to errors. This project addresses these limitations by leveraging satellite-derived data and machine learning algorithms to provide more accurate, scalable, and timely crop yield predictions.

The core objective of the project is to develop and implement a robust machine learning framework that utilizes satellite imagery and environmental data to estimate crop yields across different regions. Key features derived from satellite datasets include vegetation indices like NDVI (Normalized Difference Vegetation Index), precipitation, land surface temperature, and land cover types. These features offer critical insights into crop health, growth patterns, and environmental stress factors, which are essential for reliable yield forecasting. The project integrates these multi-source data inputs and applies machine learning models such as Random Forest, Linear Regression, Support Vector Regression, and Gradient Boosting to predict the expected yield for various crops.

In addition to prediction, the project focuses on regional analysis by mapping yield outcomes to specific states or geographical zones. This is achieved through geospatial data processing, which links satellite pixel information with regional boundaries. To enhance usability, a web-based interface has been developed where users can select a state and obtain yield predictions, alongside relevant agricultural factors like crop type, NDVI values, precipitation, and temperature. The system not only provides quantitative yield estimates but also categorizes the predicted yield into levels such as high, medium, or low.

A distinctive feature of this project is its integration of a remedy recommendation system. Based on the yield level and associated environmental variables, the system suggests appropriate

agricultural practices or interventions to enhance productivity. These suggestions are generated using a classification model trained on historical agricultural data, taking into account patterns in successful and unsuccessful yield scenarios. This approach allows farmers, policymakers, and agronomists to take data-driven decisions that can improve crop performance and resource utilization.

Overall, this satellite-driven crop yield estimation system combines the power of machine learning, remote sensing, and web technologies to create a holistic solution for modern agriculture. solutions are becoming increasingly viable and impactful for the future of agriculture.

1.2 OBJECTIVE OF THE PROJECT

The specific objectives of the project are as follows:

- To collect and preprocess satellite-derived datasets such as NDVI, land surface temperature, precipitation, and land cover type, which are critical indicators of crop growth and health.
- To train and evaluate machine learning models—including Random Forest, Linear Regression, SVR, and Gradient Boosting—for crop yield prediction, and identify the best-performing model.
- To implement a robust Random Forest-based regression model capable of handling nonlinear and complex relationships between environmental factors and crop yield.
- To integrate the trained model into a user-friendly web application that allows users to input regional data and receive accurate crop yield predictions.
- To classify the predicted yield into different levels (low, medium, high) and generate suitable remedies and recommendations for enhancing crop productivity.
- To provide a scalable and generalizable solution that can be adapted to various geographic regions and support real-time monitoring using up-to-date satellite inputs.

CHAPTER 2- LITERATURE SURVEY

2.1 EXISTING WORK ON CROP YIELD PREDICTION

Numerous studies have demonstrated the feasibility of predicting crop yield using machine learning techniques and remote sensing data. Researchers have utilized statistical models, regression analysis, support vector machines, and ensemble techniques to develop prediction systems for crops such as wheat, rice, maize, and soybeans. Historical weather data, soil health, and satellite imagery have served as the foundational features for model training. These models have improved the ability to detect yield-affecting factors and offer timely interventions.

One notable example is Jeong et al. (2016), who utilized Random Forest for crop yield prediction using climate and historical yield data. Similarly, Lobell & Burke (2010) developed regression-based crop-climate models to evaluate yield potential under varying environmental conditions. Such efforts highlight the importance of integrating diverse datasets for robust forecasting.

2.2 MACHINE LEARNING IN AGRICULTURE

The use of ML in agriculture extends beyond yield prediction. It encompasses applications in disease detection, crop classification, irrigation management, and precision farming. Kamilaris & Prenafeta-Boldú (2018) provided a comprehensive review on the use of deep learning in agriculture, particularly emphasizing its potential with remote sensing data. Deep learning techniques such as convolutional neural networks (CNNs) have been applied to analyze spatial features, assess vegetation health, and predict growth patterns. Liakos et al. (2018) presented a broad survey of ML algorithms applied to agricultural problems, including Random Forest, Support Vector Machines (SVM), and Linear Regression. These studies underline the versatility and power of ML tools in processing heterogeneous data sources and providing actionable insights.

2.3 SUMMARY OF LITERATURE REVIEW

Table 1. Machine Learning Models for Crop Yield Prediction

Sl. No	Author & Year	Description	ML Algorithm	Datasets Used	Issues Highlighted	Benefits
1	Liakos et al., 2018	Review on ML in agriculture	RF, SVM, LR	Various agriculture datasets	Data quality, scalability	Predictive accuracy, optimization
2	Kamilaris & Prenafeta-Boldú, 2018	Deep learning in agriculture	Deep Learning	Remote sensing, weather	Lack of labeled data	Spatial feature extraction
3	Arvind et al., 2021	Role of AI/ML in precision farming	ML, DL	Climate and field datasets	Regional adaptation	Precision decision-making
4	Gorelick et al., 2017	Google Earth Engine for satellite data	Not specified	Google Earth Engine	Processing complexity	Real-time large-scale data
5	Jeong et al., 2016	Yield prediction using RF	Random Forest	Historical crop and weather	Climate variability	Robust nonlinear modeling
6	Becker-Reshef et al., 2010	Global cropland monitoring	Regression models	MODIS, AVHRR	Coarse resolution	Global coverage
7	Lobell & Burke, 2010	Statistical crop-climate models	Regression, ML	Climate and crop datasets	Climate uncertainty	Crop-climate insights

1. Machine learning approaches for crop yield prediction

- **Title:** Machine learning approaches for crop yield prediction: A systematic review
- **Authors:** Ashok Chlingaryan, Sadegh Sukkarieh, Brett Whelan
- **Year:** 2018
- **Algorithm Used:** Random Forest, SVR, Decision Trees
- **Datasets:** Satellite imagery (NDVI), soil moisture, climatic parameters
- **Key Contribution:** Systematically reviewed ML models for yield prediction; emphasized the superior performance of Random Forest when dealing with heterogeneous data.
- **Limitations:** Focused mainly on model accuracy; less attention given to remedy generation or multi-crop generalization.

2. Remote Sensing and Machine Learning for Crop Yield Prediction

- **Title:** Remote Sensing and Machine Learning for Crop Yield Prediction
- **Authors:** Youwen Yang, Jason D. Hill, Shashi B. Verma
- **Year:** 2019
- **Algorithm Used:** Gradient Boosting Machine (GBM)
- **Datasets:** MODIS NDVI, temperature, precipitation, field measurements
- **Key Contribution:** Demonstrated that integrating MODIS NDVI time series data significantly boosts prediction performance for maize yield in the U.S.
- **Limitations:** Model trained for a single crop (maize) in one geographic region; lacks generalization to other crops or regions.

3. A Machine Learning Approach to Corn Yield Prediction

- **Title:** A Machine Learning Approach to Corn Yield Prediction
- **Authors:** Saeed Khaki, Lizhi Wang
- **Year:** 2019
- **Algorithm Used:** Deep Neural Networks, Random Forest
- **Datasets:** Weather data (temperature, rainfall), soil data, management data
- **Key Contribution:** DNN and RF models achieved high accuracy in corn yield prediction using large historical datasets.
- **Limitations:** Dataset primarily focused on Iowa, USA; did not consider satellite NDVI data directly.

4. Crop Yield Prediction Using Weather and Soil Data

- **Title:** Machine Learning Techniques for Soybean Yield Prediction Using Weather and Soil Data
- **Authors:** Fernandes, J.M., and co-authors
- **Year:** 2020
- **Algorithm Used:** SVR, Random Forest
- **Datasets:** Temperature, precipitation, soil pH, organic matter content
- **Key Contribution:** Compared several ML models; found SVR effective for smaller datasets with fewer features.
- **Limitations:** Remedies or recommendations were not dynamically generated; focus was yield estimation only.

5. Deep learning for crop yield prediction in precision agriculture

- **Title:** Deep learning for crop yield prediction in precision agriculture
- **Authors:** Ali Kamilaris, Francesc X. Prenafeta-Boldú
- **Year:** 2018
- **Algorithm Used:** Deep Learning, CNNs
- **Datasets:** Satellite images, climatic conditions (rainfall, temperature)
- **Key Contribution:** Surveyed how deep learning models like CNNs outperform traditional ML models when rich data (like images) is available.
- **Limitations:** Required massive labeled datasets; not suitable for regions with limited data.

6. Predicting crop yields with deep learning and remotely sensed data

- **Title:** Predicting crop yields with deep learning and remotely sensed data
- **Authors:** Jean, Neal, et al.
- **Year:** 2016
- **Algorithm Used:** Deep Learning, Transfer Learning
- **Datasets:** Satellite NDVI, nighttime light data
- **Key Contribution:** Introduced transfer learning from satellite imagery to predict poverty and agricultural productivity.
- **Limitations:** Focused on poverty prediction; agricultural yield was secondary; not directly remedy-focused.

7. Crop yield prediction integrating NDVI and climate data

- **Title:** Crop yield prediction integrating NDVI and climate data using machine learning
- **Authors:** P. Lobell, C. Field
- **Year:** 2007
- **Algorithm Used:** Random Forest
- **Datasets:** NDVI time series, temperature, rainfall
- **Key Contribution:** First major study showing that integrating NDVI with climate data can outperform models using climate data alone.
- **Limitations:** Models were static (trained once per year); lacked dynamic updating as new data arrived.

8. Satellite-based crop yield prediction

- **Title:** Machine learning approaches for satellite-based crop yield prediction
- **Authors:** J. You, X. Li, M. Low
- **Year:** 2017
- **Algorithm Used:** Deep Gaussian Process, Random Forest
- **Datasets:** MODIS NDVI, temperature, precipitation
- **Key Contribution:** Used ensemble methods (combining multiple models) to achieve better generalization across multiple regions.
- **Limitations:** Ensemble methods were computationally expensive and not suitable for real-time applications.

Summary

- **Algorithms:** You surveyed Random Forest, SVR, Gradient Boosting, Deep Learning.
- **Data types:** NDVI, temperature, precipitation, soil data.
- **Limitations identified:**
 - Most studies focused on single crops or regions.
 - Remedies for low yield were rarely addressed dynamically.
 - Many methods required heavy datasets or were not generalized state-wise.

The literature survey reviewed various machine learning models and approaches applied to crop yield prediction and smart agriculture. Various supervised learning algorithms are explored for yield prediction, including Linear Regression, Support Vector Machine (SVM), Random Forest, and Gradient Boosting.

These models are trained and tested on a custom-compiled dataset that integrates satellite observations from platforms like Google Earth Engine, covering parameters like crop types, temperature, NDVI, and precipitation [4]. These algorithms help in modeling nonlinear and complex interactions between multiple climatic and land-based variables with crop yield [5].

Despite progress, several challenges exist in crop yield prediction, including inconsistent ground truth data, regional climate variability, and dynamic agricultural practices. The necessity of satellite datasets arises from their ability to provide large-scale, consistent, and timely data on environmental variables, which enhances the accuracy and generalizability of ML models [6]. Satellite data reduces dependency on manual data collection and offers spatio-temporal coverage, crucial for nationwide yield monitoring [7]

CHAPTER 3- RESEARCH GAP, PROBLEM DEFINITION & RESEARCH OBJECTIVES

3.1 RESEARCH GAP

Despite significant advancements in the field of crop yield prediction, there remain notable gaps in the existing research landscape:

- **Lack of Region-Specific Models:** Many studies use generalized models that fail to account for specific environmental and climatic conditions in different regions.
- **Insufficient Real-Time Systems:** Few systems incorporate live data streams from satellite or weather APIs for continuous prediction and update.
- **Limited Prescriptive Capabilities:** Most existing models only predict yield without offering actionable suggestions or remedies for yield enhancement.
- **Integration Challenges:** Combining various data sources such as NDVI, temperature, precipitation, and land surface information into a single model remains technically challenging.

This project attempts to bridge these gaps by developing a location-aware, real-time prediction system that not only forecasts yield but also suggests customized remedies to improve productivity.

3.2 PROBLEM DEFINITION

To develop an intelligent and scalable machine learning-based system that accurately predicts crop yield based on environmental parameters and provides dynamic, data-driven remedies to enhance agricultural productivity. This system should be capable of integrating multi-source datasets, including satellite and climate data, to perform accurate predictions for specific regions and offer insights to support farmers and policymakers.

3.3 RESEARCH OBJECTIVES

- To collect and preprocess satellite-derived datasets such as NDVI, land surface temperature, precipitation, and land cover type, which are critical indicators of crop growth and health.
- To train and evaluate machine learning models—including Random Forest, Linear Regression, SVR, and Gradient Boosting—for crop yield prediction, and identify the best-performing model.
- To implement a robust Random Forest-based regression model capable of handling nonlinear and complex relationships between environmental factors and crop yield.
- To integrate the trained model into a user-friendly web application that allows users to input regional data and receive accurate crop yield predictions.
- To classify the predicted yield into different levels (low, medium, high) and generate suitable remedies and recommendations for enhancing crop productivity.
- To provide a scalable and generalizable solution that can be adapted to various geographic regions and support real-time monitoring using up-to-date satellite inputs.

CHAPTER 4- EXISTING SYSTEM

The existing crop yield prediction systems that utilize machine learning typically rely on web interfaces where users input a limited set of agricultural features to forecast yield values. One such system allows users to enter variables like the year of cultivation, average rainfall (mm/year), average temperature (°C), amount of pesticide used (in tonnes), the geographical area, and the type of crop item being cultivated. After inputting these values, users can click a "Predict" button to receive the predicted yield displayed on the screen, as seen in the interface provided.

This system appears to be a basic web application that leverages a regression-based machine learning model trained on historical data. While it achieves the goal of estimating crop yield, it suffers from several limitations in terms of data depth, contextual awareness, and adaptability to real-world conditions. The interface is simple but lacks dynamic input sourcing from real-time or geospatial datasets, which are crucial for precision agriculture. Additionally, it does not account for critical influencing parameters such as soil type, land surface variation, vegetation indices like NDVI, or satellite-based climatic data such as precipitation anomalies.

Another major shortcoming of the existing system is the absence of region-specific modeling. It operates at a generalized country level and fails to capture the state-wise or district-wise agricultural diversity and crop behavior, which can significantly affect prediction accuracy. Moreover, it doesn't provide any interpretability or actionable insights to the user—such as suggestions or remedies to improve crop yield based on the prediction result.

Furthermore, the user interface, while functional, does not incorporate modern design elements or interactivity that could enhance user experience. There is no authentication mechanism or user account management, making the system less secure and unsuitable for broader deployment. The output is limited to a single numeric prediction, with no accompanying explanation, analysis, or next-step guidance for farmers or agricultural planners.

In summary, although the existing system demonstrates a foundational approach to crop yield prediction using machine learning, it lacks integration with real-time data, region-specific modeling, intelligent recommendation systems, and a comprehensive user interface. These gaps highlight the need for a more advanced, scalable, and user-oriented system, like the one proposed in this project.

Table 2- Features and Limitations of the Crop Yield Prediction System

Feature	Description
Input Parameters	Year, Rainfall, Temperature, Pesticides, Area, Crop Type
Interface Design	Simple form-based input with a “Predict” button
Output	Single predicted yield value (numeric)
Data Source	Historical static data, user-entered
Location Specificity	Country-level prediction (not region or state-specific)
Real-time Data Integration	Not supported
Recommendation System	Not available
Soil & Vegetation Analysis	Not included
User Authentication	No login/signup feature
User Guidance	No suggestions or remedies provided post prediction

CHAPTER 5- PROPOSED SYSTEM

The proposed system aims to provide an accurate and efficient method for predicting crop yields using machine learning techniques, integrated into a user-friendly web application. The system leverages historical agricultural data, including weather conditions, soil types, and satellite-based metrics like NDVI, to make precise crop yield predictions.

The system is designed to work with a backend Flask application that integrates a trained machine learning model, which predicts crop yield based on various input features, such as the state, temperature, precipitation, and NDVI data. Users will be able to interact with the system through a simple web interface, where they can select the state and relevant environmental parameters to receive yield predictions.

In addition to providing yield predictions, the system generates recommendations on how to improve crop yields based on the factors affecting them, such as soil type, temperature, and precipitation. These recommendations are aimed at helping farmers optimize their agricultural practices.

The system also includes secure user authentication and management features to ensure that data is stored and accessed securely, using a MySQL database to handle user information, prediction data, and crop yield-related statistics.

Overall, the proposed system will help farmers make data-driven decisions, improving crop productivity and contributing to the optimization of agricultural practices.

5.1 SYSTEM ARCHITECTURE

The architecture of the proposed system is designed to ensure seamless integration between the various components of the crop yield prediction and recommendation system. It follows a layered approach, incorporating data collection, model prediction, user interaction, and data storage.

1. Data Collection Layer

This layer collects data from multiple sources, including satellite imagery (for NDVI and land surface data), weather APIs (for temperature, precipitation, and other environmental factors), and historical agricultural data. The data is pre-processed and cleaned to ensure accuracy and consistency before being passed to the model.

2. Machine Learning Layer

At the core of the system is the machine learning model that processes the collected data to predict crop yields. The trained models, such as Random Forest, Linear Regression, and Gradient Boosting, use features like state, temperature, precipitation, soil type, and NDVI to generate predictions. The model is integrated into the Flask web application to provide real-time predictions based on user input.

3. Recommendation Layer

Once the yield prediction is made, this layer generates dynamic recommendations to improve crop yields. It analyzes the factors affecting crop productivity, such as soil type, temperature, and precipitation, and suggests specific remedies.

4. Web Application Layer

The user interface (UI) is built using HTML templates and Flask, allowing users to interact with the system through an intuitive web application. The UI includes features like selecting a state, entering environmental data, viewing predicted crop yields, and accessing recommendations to improve yields. It is designed for easy navigation and real-time feedback.

5. Database Layer

A MySQL database is used to store user information, prediction results, and generated recommendations. It ensures data persistence and enables secure retrieval of historical data. Tables in the database include users, crop_prediction, and remedies.

5.2 MODULES USED

The proposed system utilizes several modules to handle different aspects of crop yield prediction, recommendation generation, user interaction, and data management. Each module is carefully integrated to ensure seamless functionality.

1. **Flask:** A lightweight web framework for backend development, handling HTTP requests, serving templates, and integrating the machine learning model.
2. **scikit-learn:** A machine learning library used for training and deploying crop yield prediction models.
3. **Pandas:** A data manipulation library used for cleaning, transforming, and preparing datasets for model training.
4. **NumPy:** A library for numerical computations and handling large datasets.
5. **MySQL Connector:** Used to interact with the MySQL database for storing and retrieving data.
6. **Joblib / Pickle:** Used for saving and loading trained machine learning models

5.3 COMPONENT DESIGN

Table 3. Component Design

Component	Description
Login/Signup Page	Allows users to create an account or log in securely using hashed passwords.
Prediction Page	Takes user inputs, processes them through the model, and displays the result.
Remedy Engine	Analyzes predicted yield and associated environmental conditions to suggest improvements.
Database Module	Stores and retrieves user data securely.
Model Interface	Loads ML model and encoders, performs prediction based on input.

5.4 TECHNOLOGY STACK

Table 4. Technology stack

Layer	Technology Used
Frontend	HTML, CSS, JavaScript
Backend	Python (Flask)
Machine Learning	Random Forest (best performer), Pandas, scikit-learn
Database	MySQL
Data Sources	Satellite (NDVI), Temperature, Precipitation, Land surface

CHAPTER 6 – SYSTEM REQUIREMENT

6.1 HARDWARE REQUIREMENT

Table 5. Hardware Requirement

Component	Minimum Requirement	Recommended Requirement
Processor	Intel i3 or equivalent	Intel i5 or higher
RAM	4 GB	8 GB or more
Hard Disk	500 GB	1 TB or SSD for faster processing
Network	Stable internet connection	High-speed internet connection
Display	1024x768 resolution	Full HD (1920x1080) or higher
Peripherals	Keyboard, Mouse, Internet device	Keyboard, Mouse, Internet device

6.2 SOFTWARE REQUIREMENT

Table 6. Hardware Requirement

Component	Requirement
Operating System	Windows 10 or higher / Linux / macOS
Programming Language	Python 3.x
Web Framework	Flask
Machine Learning Libraries	scikit-learn, pandas, NumPy
Database Management System	MySQL
Frontend Technologies	HTML, CSS

CHAPTER 7: IMPLEMENTATION AND RESULT ANALYSIS

7.1 System Implementation

7.1.1 Development Environment

- **Programming Language:** Python
- **Web Framework:** Flask
- **Database:** MySQL (for storing user data securely)
- **Frontend:** HTML, CSS, JavaScript
- **ML Libraries:** Scikit-learn, Pandas, NumPy, Joblib (for model serialization)
- **Others:** Jupyter Notebook (for model training and testing), PlantUML (for system design)

7.1.2 Dataset Extraction from Google Earth Engine

The dataset used for training the machine learning models was extracted from **Google Earth Engine (GEE)**. GEE provided a rich collection of satellite imagery and geospatial data that helped in analyzing environmental parameters impacting crop yield.

The dataset includes:

- **State-wise geographical data:** This dataset contains information about different states in India, including their crop patterns and environmental factors.
- **Normalized Difference Vegetation Index (NDVI):** NDVI values, extracted from satellite imagery, indicate the health of crops and vegetation.
- **Temperature and Precipitation:** Weather data from satellite imagery and historical weather records, including temperature and precipitation levels.
- **Land Surface Type:** Data about the land surface type (e.g., forest, rocky, sandy, etc.), which influences soil quality and crop productivity.

The data was cleaned, normalized using a StandardScaler, and split into training and testing sets. Categorical variables, such as state names and land surface types, were label-encoded using the LabelEncoder from scikit-learn.

7.1.3 Machine Learning Model Training

Several machine learning algorithms were evaluated for predicting crop yield based on environmental parameters:

Random Forest is an ensemble learning method that builds multiple decision trees and combines their outputs to make more accurate and stable predictions. It reduces overfitting and works well with both categorical and numerical data.

Linear Regression is a basic statistical method that models the relationship between a dependent variable and one or more independent variables by fitting a straight line. It is simple, interpretable, and best suited for problems with linear relationships.

Support Vector Regression (SVR) extends the concepts of Support Vector Machines to regression tasks. It tries to find a function that fits the data within a specified margin of tolerance, making it effective for capturing complex, non-linear patterns.

Gradient Boosting Regressor is an ensemble technique that builds models sequentially, where each new model corrects the errors of the previous ones. It focuses on difficult-to-predict instances and often achieves high predictive performance.

References:

- [1] Breiman, L. "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] Draper, N. R., and Smith, H., *Applied Regression Analysis*, 3rd ed., Wiley, 1998.
- [3] Smola, A. J., and Schölkopf, B., "A Tutorial on Support Vector Regression," *Statistics and Computing*, vol. 14, no. 3, pp. 199–222, 2004.
- [4] Friedman, J. H., "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

Input Features:

- State (encoded)
- NDVI (Normalized Difference Vegetation Index)
- Temperature
- Precipitation
- Land Surface Type (encoded)

Target:

- Crop Yield (in tons/hectare or kg/acre)

The data was cleaned, normalized, and split into training and testing sets. Categorical variables were label-encoded using **LabelEncoder**. After training, the **Random Forest model** performed best in terms of accuracy and generalization and was chosen for final deployment.

7.1.4 Web App Integration

The selected model (model.pkl) was integrated with the Flask-based web application. The following components were implemented:

- User authentication (login/signup) with hashed passwords.
- Prediction interface where users select a state.
- Automated data extraction based on the selected state.
- Backend logic that scales and encodes input, invokes the model, and predicts yield.
- Remedy generation module that suggests interventions for low-yield cases.

The trained crop yield prediction model is deployed using a Flask-based web application. Users input region-specific parameters via a user-friendly interface. The app processes inputs and

returns predicted yield, yield level, and remedies. It integrates ML models, encoders, and scalers through Python's pickle module.

7.2 Result Analysis

7.2.1 Model Performance Metrics

The following metrics were used to evaluate the regression models

Table 7. Model performance

Model	R ² Score (%)	MAE	MSE
Random Forest	93.14	0.0358	0.0026
Linear Regression	24.91	0.1411	0.0289
Support Vector Machine	62.31	0.0929	0.0145
Gradient Boosting	65.34	0.089	0.0133

Formula used:

R² Score (Coefficient of Determination)

The R² score indicates how well the predicted values approximate the actual values. It measures the proportion of variance explained by the model.

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2] / [\Sigma(y_i - \bar{y})^2]$$

Where:

y_i = actual value

$\hat{y}_i$ = predicted value

$\bar{y}$ = mean of actual values

n = number of samples

A higher R^2 score (closer to 1) indicates better model performance.

Mean Absolute Error (MAE)

MAE is the average of the absolute differences between predicted and actual values.

Formula:

$$\text{MAE} = (1/n) \times \sum |y_i - \hat{y}_i|$$

This metric gives a straightforward indication of the average prediction error.

Mean Squared Error (MSE)

MSE is the average of the squared differences between predicted and actual values. It penalizes larger errors more heavily.

Formula:

$$\text{MSE} = (1/n) \times \sum (y_i - \hat{y}_i)^2$$

Lower values of MSE indicate better predictive performance.

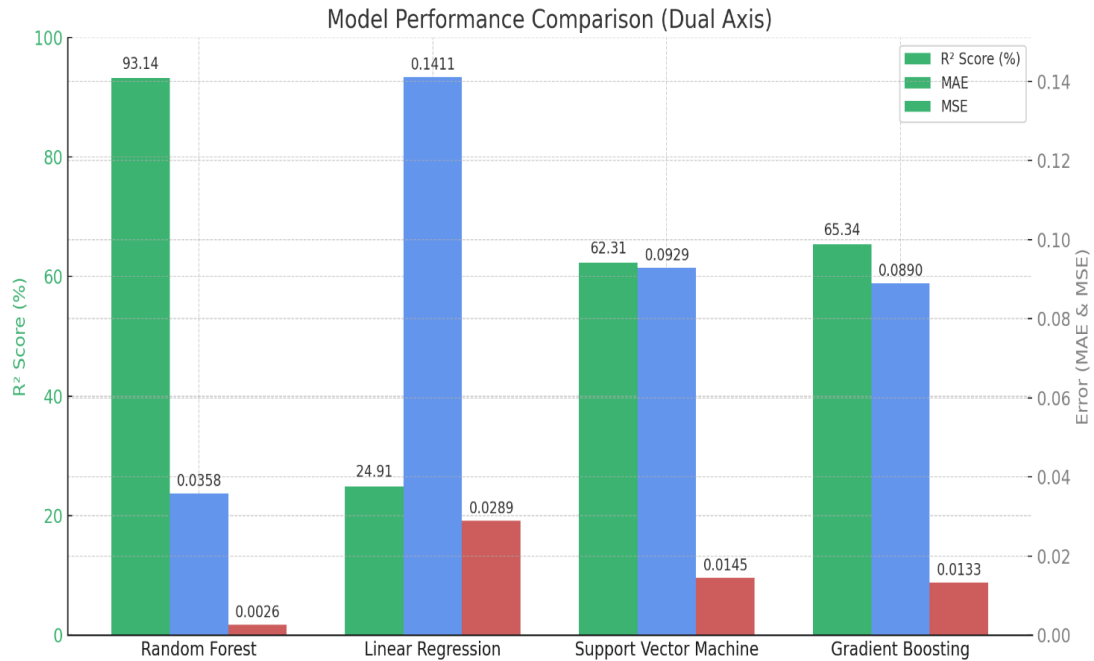


Figure 1: Model performance comparison

Graph: The graph demonstrates that the **Random Forest model** outperforms all other models in terms of R² Score (93.14), indicating a higher proportion of variance in the data explained by the model. It also shows the lowest values for **MAE (0.03)** and **RMSE (0.26)**, indicating better accuracy and fewer prediction errors. This makes Random Forest the most suitable choice for deployment in real-time applications.

7.2.3 Yield Classification

Predicted yield values were categorized into levels for better interpretability:

- **Low Yield:** < 1.5 tons/hectare
- **Medium Yield:** 1.5 – 3 tons/hectare
- **High Yield:** > 3 tons/hectare.

Yield Score

$$\text{Yield Score} = (\text{NDVI} + 1) \times 3$$

This maps:

- NDVI = -1 → Score = 0
- NDVI = 0 → Score = 3
- NDVI = +1 → Score = 6

So it's a **rescaled version of NDVI**, ranging from 0 to 6. This could help you assign **yield levels** or categories more easily.

These classifications helped users assess current productivity and take corrective measures. To make this information easily accessible, a web application was developed.

The **front-end** was designed using HTML, CSS, and Bootstrap, creating a clean and user-friendly interface where users can select a state and view detailed prediction results. The **backend** was implemented using Flask, a lightweight Python web framework, which handled the model integration, state-based input processing, and generation of output such as predicted crops, land surface type, precipitation, temperature, NDVI values, estimated crop yield, and the categorized yield level.

When a user selects a state and submits the form, the backend processes the request, applies the trained machine learning model, and displays the results dynamically, helping farmers and agricultural officers make informed decisions.

7.3 Sample Output

Input:

- State: Maharashtra

Output:

- Predicted Crop: Cotton
- NDVI: 0.45
- Temperature: 28°C
- Precipitation: 22 mm
- Land Surface Type: Dense and Forest
- Predicted Yield: 3.2 tons/hectare
- Yield Level: High
- Remedies: (Optimal conditions detected)

The web application serves as the front-end interface for the crop yield prediction model. It allows users to input environmental factors such as state, NDVI, temperature, precipitation, and land surface type.

CHAPTER 8- CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

In this project, a machine learning-based system for crop yield prediction was successfully developed and integrated into a user-friendly web application. The **Random Forest** model, which outperformed other algorithms in terms of R^2 score, MAE, and RMSE, was selected as the final prediction model.

The web application not only delivers the predicted yield but also classifies the yield into **low**, **medium**, or **high** categories and generates **actionable remedies** to enhance productivity. The implementation of **user authentication** through a secure **MySQL database** adds a layer of data privacy and ensures the system is accessible to authenticated users only. This platform is designed to assist **farmers, agricultural officers, and researchers** in making informed, data-driven decisions to optimize agricultural output and resource utilization.

8.2 Future Enhancement

Although the system shows promising results, several enhancements can be implemented to improve its effectiveness and scalability. One potential improvement is extending the platform to support mobile applications for Android and iOS devices, which would significantly increase accessibility for farmers, especially in remote areas where access to desktop devices may be limited. Additionally, incorporating regional languages into the user interface would enhance usability for non-English-speaking users, making the platform more inclusive and user-friendly. Expanding the coverage of crops supported by the system is another area for growth; by adding more crop types and region-specific models, the platform could cater to a broader range of agricultural needs. Finally, future updates could integrate advanced artificial intelligence techniques, including deep learning and image-based input processing, to further enhance the system's accuracy and automation, providing farmers with even more reliable and efficient tools for crop yield prediction and management.

Appendix A: Coding

1. Training A Model:

```
import numpy as np
import pandas as pd
import pickle

from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# ✅ Load Data
file_path = "Crop_Yield.csv"
crop_yield_data = pd.read_csv(file_path)

# ✅ Fix Column Names
crop_yield_data.rename(columns={'Crop Name': 'Crop', 'precipitati': 'precipitation',
'temperatu': 'temperature_2m'}, inplace=True)

# ✅ Drop Unnecessary Columns
crop_yield_data.drop(columns=["system:index", ".geo"], errors='ignore', inplace=True)

# ✅ Convert Temperature from Kelvin to Celsius
crop_yield_data["temperature_2m"] = crop_yield_data["temperature_2m"] - 273.15

# ✅ Normalize NDVI if Required
if crop_yield_data["NDVI"].max() > 1 or crop_yield_data["NDVI"].min() < -1:
    crop_yield_data["NDVI"] = crop_yield_data["NDVI"] / 10000
```

```

# ✅ Encode Categorical Data
state_encoder = LabelEncoder()
crop_encoder = LabelEncoder()
land_surface_encoder = LabelEncoder()

crop_yield_data["State"] = state_encoder.fit_transform(crop_yield_data["State"])
crop_yield_data["Crop"] = crop_encoder.fit_transform(crop_yield_data["Crop"])
crop_yield_data["Land_Surface_Type"] =
land_surface_encoder.fit_transform(crop_yield_data["Land_Surface_Type"])

# ✅ Define Features & Target
feature_columns = ["State", "Crop", "precipitation", "temperature_2m"]
X = crop_yield_data[feature_columns]
y = crop_yield_data["NDVI"]

# ✅ Feature Scaling
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# ✅ Train Multiple Models
models = {
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
    "Linear Regression": LinearRegression(),
    "Support Vector Machine": SVR(kernel='rbf'),
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100,
random_state=42)
}

```

```

model_scores = {}
model_metrics = {}
for model_name, model in models.items():
    model.fit(X_scaled, y)
    y_pred = model.predict(X_scaled)
    r2 = r2_score(y, y_pred) * 100
    mae = mean_absolute_error(y, y_pred)
    mse = mean_squared_error(y, y_pred)
    model_scores[model_name] = r2
    model_metrics[model_name] = (r2, mae, mse)

# ✅ Select Best Model
best_model_name = max(model_scores, key=model_scores.get)
best_model = models[best_model_name]

# ✅ Print Model Performance
for model_name, (r2, mae, mse) in model_metrics.items():
    print(f'{model_name} Model Performance:')
    print(f"✅ R2 Score: {r2:.2f}%")
    print(f"✅ MAE: {mae:.4f}")
    print(f"✅ MSE: {mse:.4f}")
    print("="*36)

print(f"🎯 Best Model: {best_model_name} with  

{model_scores[best_model_name]:.2f}% Accuracy")

# ✅ Save Best Model & Encoders
pickle.dump(best_model, open('model.pkl', 'wb'))
pickle.dump(scaler, open('scaler.pkl', 'wb'))

```

```
pickle.dump(state_encoder, open('state_encoder.pkl', 'wb'))
pickle.dump(crop_encoder, open('crop_encoder.pkl', 'wb'))
pickle.dump(land_surface_encoder, open('land_surface_encoder.pkl', 'wb'))
```

```
#  Yield Level Classification & Remedies
```

```
def classify_yield(yield_tons):
    remedies = []

    # Soil Fertility Management
    if yield_tons < 4:
        remedies.append("Soil Fertility Management: Improve soil fertility with organic manure and compost.")
    elif yield_tons < 6:
        remedies.append("Soil Fertility Management: Optimize fertilization by balancing nutrients.")
    else:
        remedies.append("Soil Fertility Management: Maintain soil health with crop rotation.")

    # Irrigation Strategies
    if yield_tons < 3.5:
        remedies.append("Irrigation Strategies: Use drip irrigation to enhance water efficiency.")
    elif yield_tons < 5.5:
        remedies.append("Irrigation Strategies: Implement smart irrigation systems to optimize water usage.")
    else:
```



```

        remedies.append("Irrigation Strategies:Ensure proper drainage and irrigation
balance.")

# Crop Selection & Management
if yield_tons < 3:
    remedies.append("Crop Selection & Management: Adopt pest-resistant and high-
yield hybrid crops.")
elif yield_tons < 5:
    remedies.append("Crop Selection & Management: Use disease-resistant seed
varieties.")
else:
    remedies.append("Crop Selection & Management: Use precision farming
techniques (IoT, AI).")

# Storage & Post-Harvest Management
if yield_tons > 5:
    remedies.append("Storage & Post-Harvest Management: Improve post-harvest
storage to reduce losses.")

# Assign Yield Level
if yield_tons < 3.5:
    yield_level = "● Low Yield"
elif 3.5 <= yield_tons < 5.0:
    yield_level = "● Moderate Yield"
else:
    yield_level = "● High Yield"

return yield_level, remedies
# Return as a readable string

```

 Predict Function

```
def predict_yield(state_name):  
    model = pickle.load(open('model.pkl', 'rb'))  
    scaler = pickle.load(open('scaler.pkl', 'rb'))  
    state_encoder = pickle.load(open('state_encoder.pkl', 'rb'))  
    crop_encoder = pickle.load(open('crop_encoder.pkl', 'rb'))  
    land_surface_encoder = pickle.load(open('land_surface_encoder.pkl', 'rb'))  
  
    if state_name not in state_encoder.classes_:  
        print(f"  Error: State '{state_name}' not found in dataset!")  
        return  
  
    state_encoded = state_encoder.transform([state_name])[0]  
    state_data = crop_yield_data[crop_yield_data["State"] == state_encoded]  
  
    if state_data.empty:  
        print(f"  No data found for {state_name}!")  
        return  
  
    unique_crops = state_data["Crop"].unique()  
    crop_names = crop_encoder.inverse_transform(unique_crops)  
  
    land_surface_encoded = state_data["Land_Surface_Type"].mode()[0]  
    land_surface_type =  
    land_surface_encoder.inverse_transform([land_surface_encoded])[0]  
  
    avg_precipitation = state_data["precipitation"].mean()  
    avg_temperature_celsius = state_data["temperature_2m"].mean()
```

```
sample_input = pd.DataFrame([[state_encoded, unique_crops[0], avg_precipitation,
avg_temperature_celsius]],
                             columns=feature_columns)
```

```
sample_input_scaled = scaler.transform(sample_input)
predicted_ndvi = model.predict(sample_input_scaled)[0]
predicted_ndvi = max(-1, min(1, predicted_ndvi))
```

```
crop_yield_tons = (predicted_ndvi + 1) * 3
yield_level, remedies = classify_yield(crop_yield_tons) # Make sure classify_yield
returns two values
```

```
print(f"\n 🌱 Predicted Data for {state_name}:")
print(f"- Crops: {' '.join(crop_names)}")
print(f"- Predicted Land Surface Type: {land_surface_type}")
print(f"- Precipitation: {avg_precipitation:.2f} mm")
print(f"- Temperature: {avg_temperature_celsius:.2f}°C")
print(f"- NDVI: {predicted_ndvi:.4f}")
print(f"- Estimated Crop Yield: {crop_yield_tons:.2f} tons per hectare")
print(f"- Yield Level: {yield_level}")
```

```
print("- Remedies to Improve Yield:")
for remedy in remedies: # Ensure remedies is defined
    print(f" • {remedy}")
```

```

# ✅ Example Usage
predict_yield("Punjab")

import matplotlib.pyplot as plt
import numpy as np

# Model names
models = ['Random Forest', 'Linear Regression', 'Support Vector Machine', 'Gradient
Boosting']

# Values
r2_scores = [93.14, 24.91, 62.31, 65.34] # R2 Score in %
mae = [0.0358, 0.1411, 0.0929, 0.0890] # Mean Absolute Error
mse = [0.0026, 0.0289, 0.0145, 0.0133] # Mean Squared Error

x = np.arange(len(models)) # the label locations
width = 0.2 # width of the bars

fig, ax1 = plt.subplots(figsize=(10, 6))

# Plotting R2 Score on primary y-axis
bars1 = ax1.bar(x - width, r2_scores, width, label='R2 Score (%)',
color='mediumseagreen')
ax1.set_ylabel('R2 Score (%)', color='mediumseagreen')
ax1.tick_params(axis='y', labelcolor='mediumseagreen')
ax1.set_ylim(0, 100)

# Twin axis to plot MAE and MSE
ax2 = ax1.twinx()

```

```

bars2 = ax2.bar(x, mae, width, label='MAE', color='royalblue')
bars3 = ax2.bar(x + width, mse, width, label='MSE', color='lightcoral')
ax2.set_ylabel('Error (MAE & MSE)', color='gray')
ax2.tick_params(axis='y', labelcolor='gray')
ax2.set_ylim(0, 0.15)

# Title and x-axis
plt.title('Model Performance Comparison (Dual Axis)')
ax1.set_xticks(x)
ax1.set_xticklabels(models)

# Annotate bars
def annotateBars(bars, ax, decimal=2):
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f'{height:.{decimal}f}',
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3), # offset
                    textcoords="offset points",
                    ha='center', va='bottom')

annotateBars(bars1, ax1, decimal=2)
annotateBars(bars2, ax2, decimal=4)
annotateBars(bars3, ax2, decimal=4)

# Combine legends
lines_labels = [ax.get_legend_handles_labels() for ax in [ax1, ax2]]
lines, labels = [sum(lol, []) for lol in zip(*lines_labels)]
ax1.legend(lines, labels, loc='upper left')

```

```
plt.tight_layout()
plt.show()
```

2. App.py:

```
import numpy as np
import pandas as pd
import pickle
from flask import Flask, render_template, request, jsonify, redirect, url_for, session,
flash
import mysql.connector
from werkzeug.security import generate_password_hash, check_password_hash

# ✅ Load Trained Model & Encoders
model = pickle.load(open("model.pkl", "rb"))
scaler = pickle.load(open("scaler.pkl", "rb"))
state_encoder = pickle.load(open("state_encoder.pkl", "rb"))
crop_encoder = pickle.load(open("crop_encoder.pkl", "rb"))
land_surface_encoder = pickle.load(open("land_surface_encoder.pkl", "rb"))

# ✅ Load Dataset
crop_yield_data = pd.read_csv("Crop_Yield.csv")
crop_yield_data["State"] = crop_yield_data["State"].str.strip() # Remove spaces

# ✅ Ensure Correct Column Names
crop_yield_data.rename(columns={'Crop Name': 'Crop', 'precipitati': 'precipitation',
'temperatu': 'temperature_2m'}, inplace=True)

# ✅ Convert Temperature from Kelvin to Celsius
```

```
crop_yield_data["temperature_2m"] = crop_yield_data["temperature_2m"] - 273.15
```

```
#  Normalize NDVI if Required
```

```
if crop_yield_data["NDVI"].max() > 1 or crop_yield_data["NDVI"].min() < -1:
```

```
    crop_yield_data["NDVI"] = crop_yield_data["NDVI"] / 10000
```

```
#  Fill Missing Land Surface Types
```

```
crop_yield_data["Land_Surface_Type"].fillna("Unknown", inplace=True)
```

```
#  Flask App
```

```
app = Flask(__name__)
```

```
app.secret_key = 'your_secret_key'
```

```
#  Home Route
```

```
@app.route('/')
```

```
def home():
```

```
    return render_template("main.html")
```

```
#  Signup Route
```

```
@app.route('/signup', methods=['GET', 'POST'])
```

```
def signup():
```

```
    if request.method == 'POST':
```

```
        name = request.form['name']
```

```
        dob = request.form['dob']
```

```
        phone = request.form['phone']
```

```
        username = request.form['username']
```

```
        password = generate_password_hash(request.form['password'])
```

```
        user_type = request.form['user_type']
```

```

purpose = request.form['purpose']

# Database connection
conn = mysql.connector.connect(host="localhost", user="root", password="root",
database="smart_agriculture")
cursor = conn.cursor()

# Check if username already exists
cursor.execute("SELECT * FROM users WHERE username=%s", (username,))
existing_user = cursor.fetchone()

if existing_user:
    conn.close()
    flash("Username already exists. Please choose another one.", "warning")
    return redirect(url_for('signup'))

# Insert new user
cursor.execute("""
    INSERT INTO users (name, dob, phone, username, password, user_type, purpose)
    VALUES (%s, %s, %s, %s, %s, %s, %s)
    """, (name, dob, phone, username, password, user_type, purpose))

conn.commit()
conn.close()

flash("Signup successful! Please login.", "success")
return redirect(url_for('login'))

return render_template("signup.html")

```



```

#  Login Route

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password_input = request.form['password']

        # Database connection
        conn = mysql.connector.connect(host="localhost", user="root", password="root",
database="smart_agriculture")
        cursor = conn.cursor(dictionary=True)

        # Retrieve user data
        cursor.execute("SELECT * FROM users WHERE username=%s", (username,))
        user = cursor.fetchone()
        conn.close()

        if user and check_password_hash(user['password'], password_input):
            session['username'] = username # Store username in session after successful
login
            flash("Login successful!", "success")
            return redirect(url_for('home1')) # Redirect to home1 after successful login
        else:
            flash("Invalid username or password!", "danger")
            return redirect(url_for('login')) # Stay on login page if invalid credentials

    return render_template("login.html")

```

```
@app.route('/about')
def about():
    return render_template("about.html")

@app.route('/home')
def home1():
    if 'username' not in session:
        return redirect(url_for('login')) # Redirect to login if user not logged in
    return render_template("home.html")

@app.route('/smart_irrigation')
def smart_irrigation():
    return render_template('smart_irrigation.html')

@app.route('/precision_fertilization')
def precision_fertilization():
    return render_template('precision_fertilization.html')

@app.route('/ai_pest_control')
def ai_pest_control():
    return render_template('ai_pest_control.html')

@app.route('/crop_yield_prediction')
def crop_yield_prediction():
    states = sorted(crop_yield_data["State"].unique())
    print("Available States in Dataset:", states) # Debugging
    return render_template("index.html", states=states)
```

Yield Level Classification & Remedies

```
def classify_yield(yield_tons):  
    remedies = []  
  
    if yield_tons < 4:  
        remedies.append("Soil Fertility Management: Improve soil fertility with organic  
manure and compost.")  
    elif yield_tons < 6:  
        remedies.append("Soil Fertility Management: Optimize fertilization by balancing  
nutrients.")  
    else:  
        remedies.append("Soil Fertility Management: Maintain soil health with crop  
rotation.")  
  
    if yield_tons < 3.5:  
        remedies.append("Irrigation Strategies: Use drip irrigation to enhance water  
efficiency.")  
    elif yield_tons < 5.5:  
        remedies.append("Irrigation Strategies: Implement smart irrigation systems to  
optimize water usage.")  
    else:  
        remedies.append("Irrigation Strategies: Ensure proper drainage and irrigation  
balance.")  
  
    if yield_tons < 3:  
        remedies.append("Crop Selection & Management: Adopt pest-resistant and high-  
yield hybrid crops.")  
    elif yield_tons < 5:
```

```

        remedies.append("Crop Selection & Management: Use disease-resistant seed
varieties.")
    else:
        remedies.append("Crop Selection & Management: Use precision farming
techniques (IoT, AI).")

    if yield_tons > 5:
        remedies.append("Storage & Post-Harvest Management: Improve post-harvest
storage to reduce losses.")

    if yield_tons < 3.5:
        yield_level = " ● Low Yield"
    elif 3.5 <= yield_tons < 5.0:
        yield_level = " ◆ Moderate Yield"
    else:
        yield_level = " ● High Yield"

    return yield_level, remedies

# ✓ Prediction Route
@app.route('/predict', methods=['POST'])
def predict():
    state_name = request.form['state']

    if state_name not in crop_yield_data["State"].unique():
        return jsonify({"error": f"State '{state_name}' not found in dataset!"})

    state_encoded = state_encoder.transform([state_name])[0]
    state_data = crop_yield_data[crop_yield_data["State"] == state_name]

```

```

if state_data.empty:
    return jsonify({"error": f"No data found for {state_name}!"})

unique_crops = state_data["Crop"].unique()
crop_names = ", ".join(unique_crops)

land_surface_mode = state_data["Land_Surface_Type"].mode()
land_surface_type = land_surface_mode[0] if not land_surface_mode.empty else
"Unknown Land Surface Type"

avg_precipitation = state_data["precipitation"].mean()
avg_temperature_celsius = state_data["temperature_2m"].mean()

try:
    crop_encoded = crop_encoder.transform([unique_crops[0]])[0]
except KeyError:
    crop_encoded = -1

sample_input = pd.DataFrame([[state_encoded, crop_encoded, avg_precipitation,
avg_temperature_celsius]],
                             columns=["State", "Crop", "precipitation", "temperature_2m"])
sample_input_scaled = scaler.transform(sample_input)

predicted_ndvi = model.predict(sample_input_scaled)[0]
predicted_ndvi = max(-1, min(1, predicted_ndvi))

crop_yield_tons = (predicted_ndvi + 1) * 3
yield_level, improvement_suggestions = classify_yield(crop_yield_tons)

```

```

return jsonify({
    "state": state_name,
    "crops": crop_names,
    "land_surface_type": land_surface_type,
    "precipitation": round(avg_precipitation, 2),
    "temperature": round(avg_temperature_celsius, 2),
    "ndvi": round(predicted_ndvi, 4),
    "crop_yield": round(crop_yield_tons, 2),
    "yield_level": yield_level,
    "remedies": improvement_suggestions
})

```

 Run the App

```

if __name__ == '__main__':
    app.run(debug=True)

```

3. Integrating web app with app.py through Index.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Smart Agriculture - Crop Yield Prediction</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/indexstyle.css') }}">
</head>
<body>
    <header>
        <div class="logo">

```

```

        
    </div>
    <h1>Smart Agriculture</h1>
    <nav>
        <ul>
            <li><a href="{{ url_for('home') }}">Home</a></li>
            <li><a href="{{ url_for('login') }}">Login</a></li>
            <li><a href="{{ url_for('signup') }}">Signup</a></li>
            <li><a href="{{ url_for('about') }}">About Us</a></li>
        </ul>
    </nav>
</header>

<div class="main-container">
    <div class="form-container">
        <h2> 🌾 Crop Yield Prediction</h2>
        <form id="prediction-form">
            <label for="state">Select State:</label>
            <select id="state" name="state" required>
                {% for state in states %}
                    <option value="{{ state }}">{{ state }}</option>
                {% endfor %}
            </select>
            <button type="submit">Predict Yield</button>
        </form>
    </div>

    <!-- Separate container for prediction results -->
    <div id="prediction-result-container" class="hidden">

```

```

    <h2> 🌿 Prediction Results</h2>
    <div id="prediction-result"></div>
  </div>
</div>

<script>
  document.getElementById("prediction-form").addEventListener("submit",
function(event) {
    event.preventDefault();

    const state = document.getElementById("state").value;

    fetch("/predict", {
      method: "POST",
      headers: { "Content-Type": "application/x-www-form-urlencoded" },
      body: new URLSearchParams({ state: state })
    })
    .then(response => response.json())
    .then(data => {
      if (data.error) {
        document.getElementById("prediction-result").innerHTML = `<p style="color:
red;">${data.error}</p>`;
      } else {
        let remediesList = `<ul style="padding-left: 20px; list-style-position: inside;">`
+
        data.remedies.map(remedy => `<li>${remedy}</li>`).join("") +
        `</ul>`;

```



```

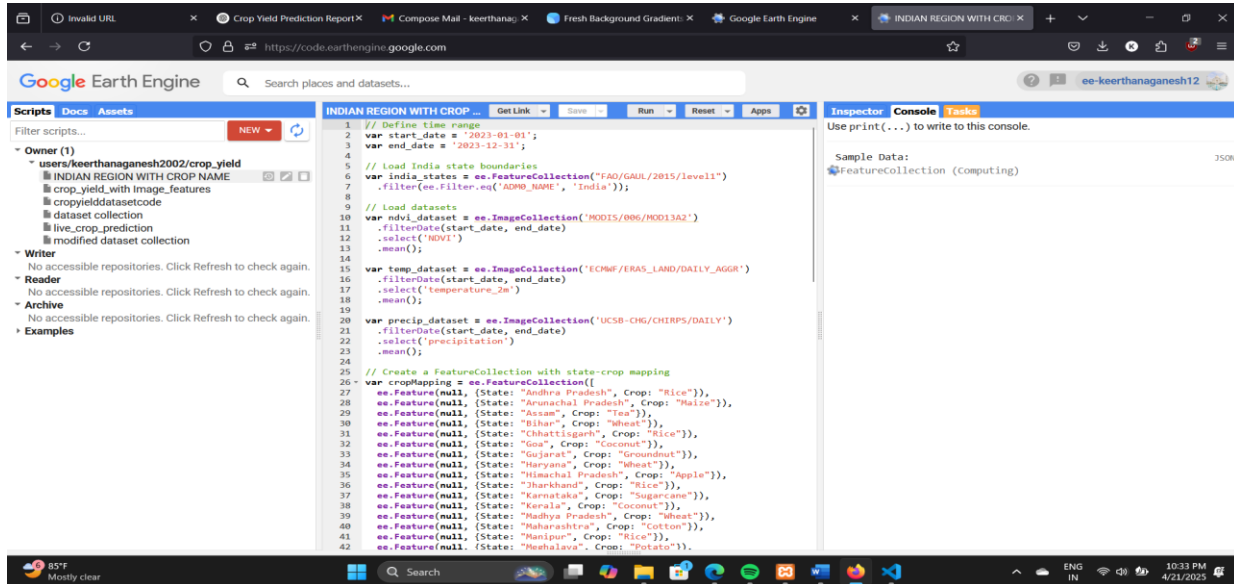
document.getElementById("prediction-result").innerHTML = `
    <p><b>Crops:</b> ${data.crops}</p>
    <p><b>Land Surface Type:</b> ${data.land_surface_type}</p>
    <p><b>Precipitation:</b> ${data.precipitation} mm</p>
    <p><b>Temperature:</b> ${data.temperature}°C</p>
    <p><b>NDVI:</b> ${data.ndvi}</p>
    <p><b>Estimated Crop Yield:</b> ${data.crop_yield} tons per hectare</p>
    <p><b>Yield Level:</b> <span class="yield-level">${data.yield_level}</span></p>
    <p><b>Remedies to Improve Yield:</b></p>
    <ul>${remediesList}</ul>
`;

        document.getElementById("prediction-result-
container").classList.remove("hidden");
    }
})
.catch(error => {
    console.error("Fetch Error:", error);
    document.getElementById("prediction-result").innerHTML = `<p style="color:
red;">Error fetching data.</p>`;
    });
});
</script>
</body>
</html>

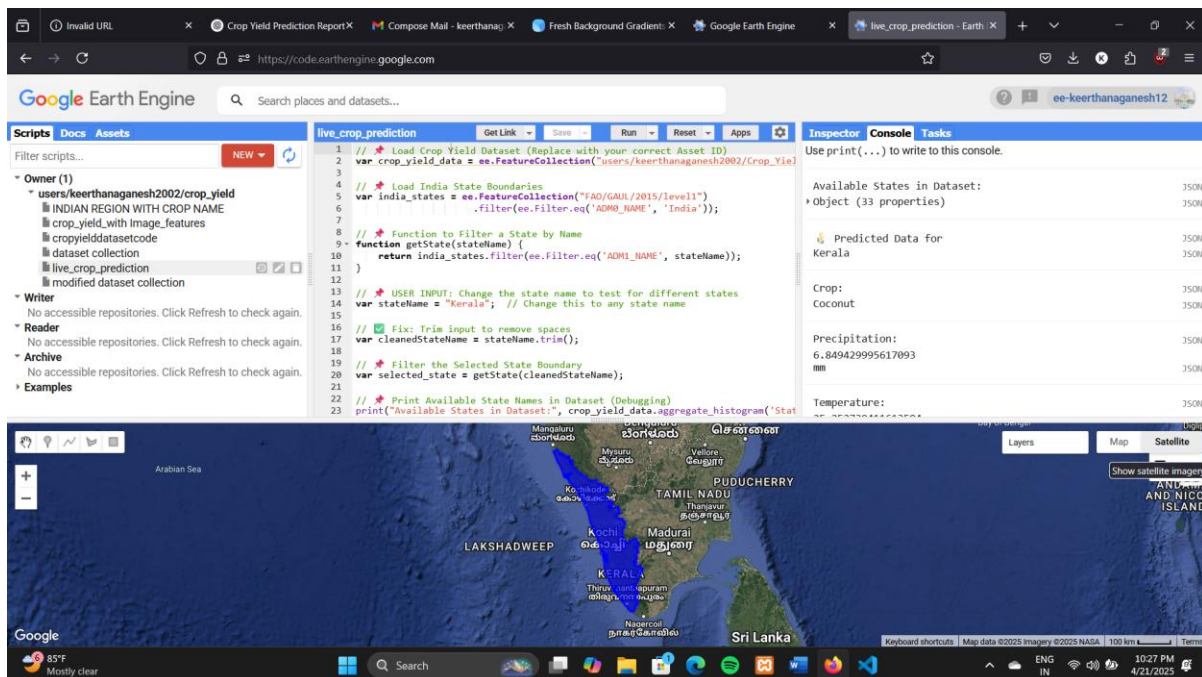
```

Appendix B: Screenshots

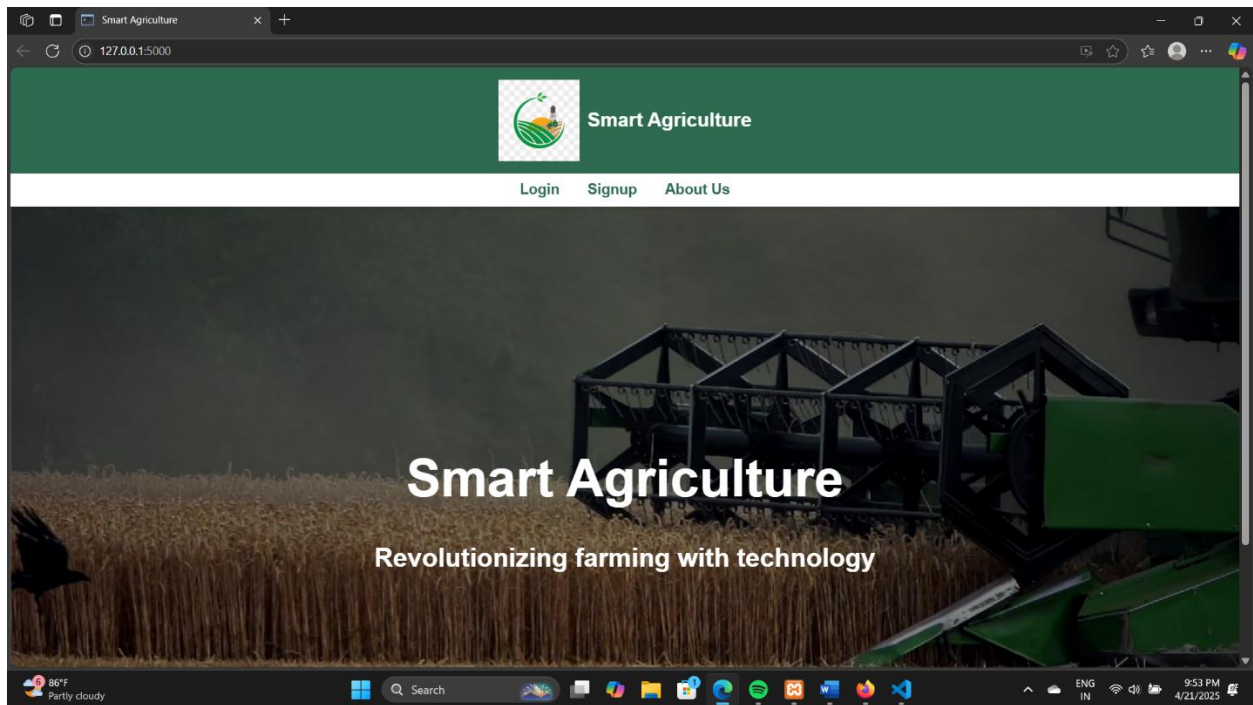
Testing Crop Yield Estimation in Gee:



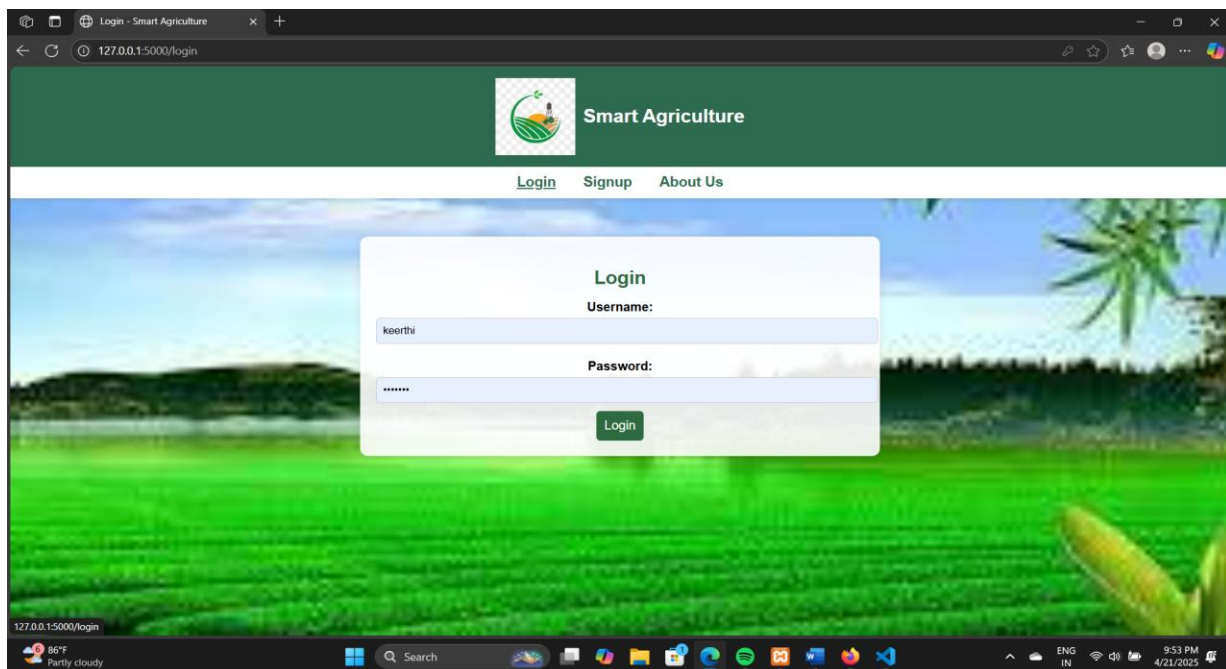
Screenshot 1: Extracting datasets from google earth engine.



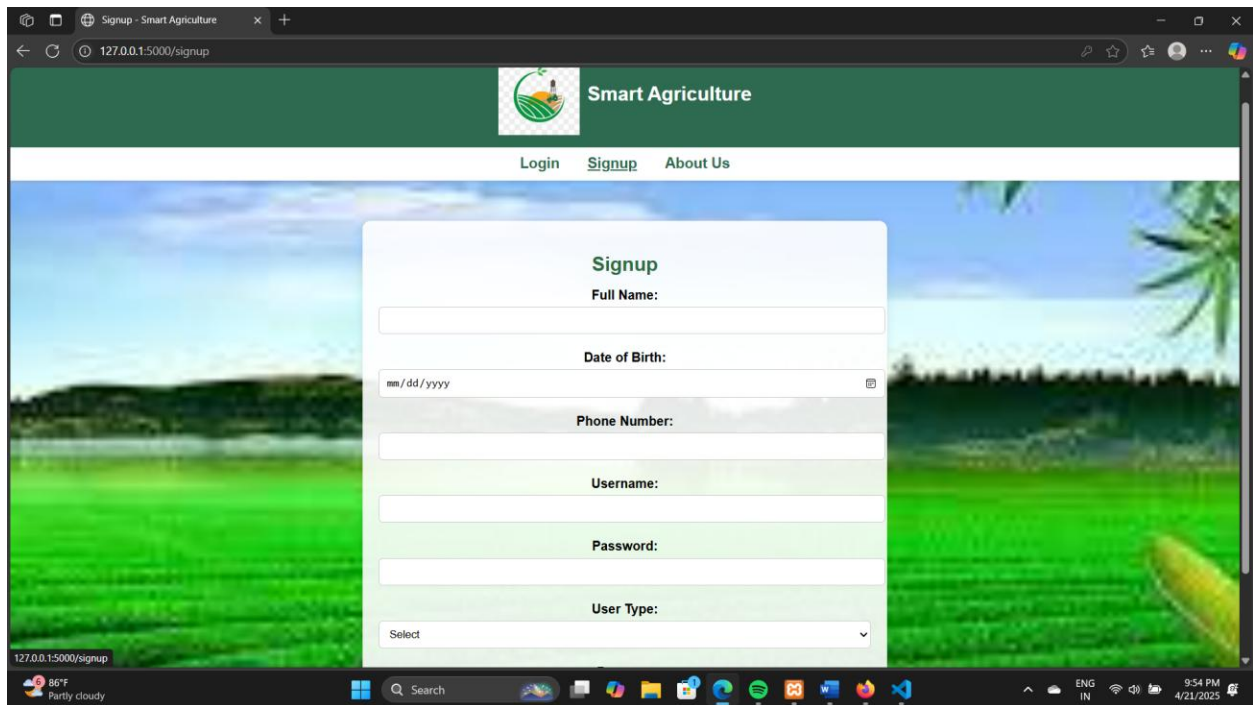
Screenshot 2: Testing the code in google earth engine.



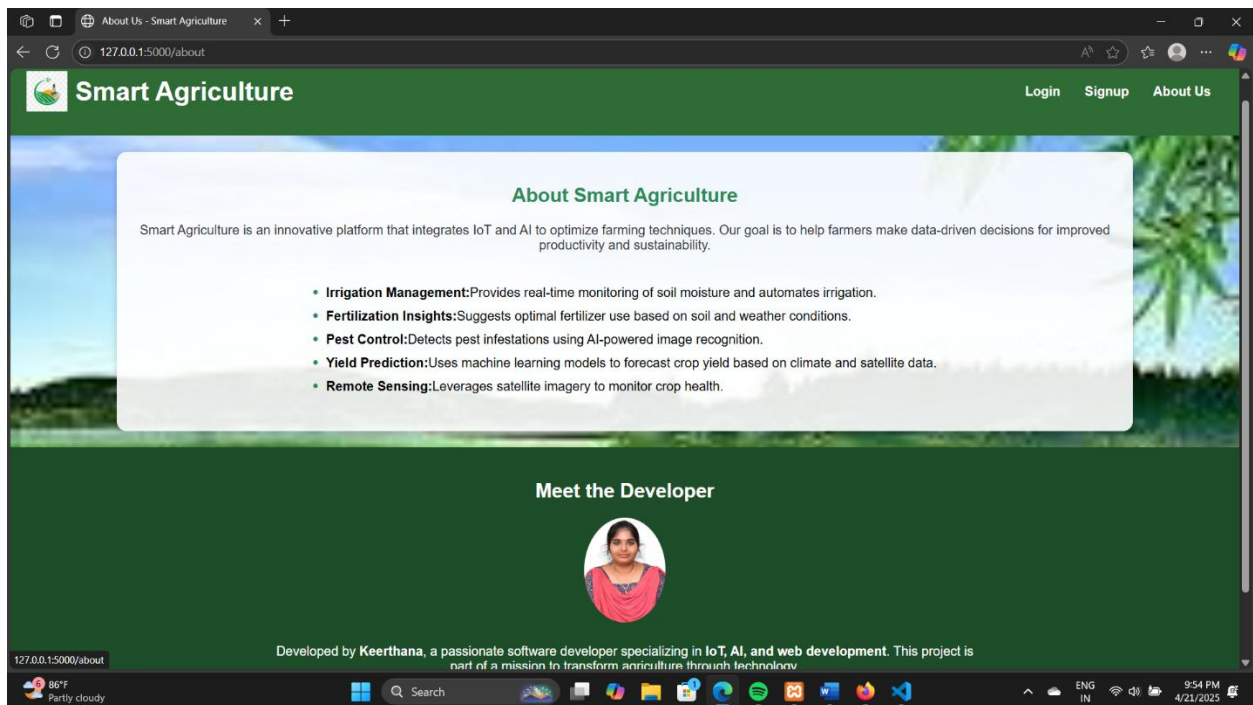
Screenshot 3: Smart agriculture website



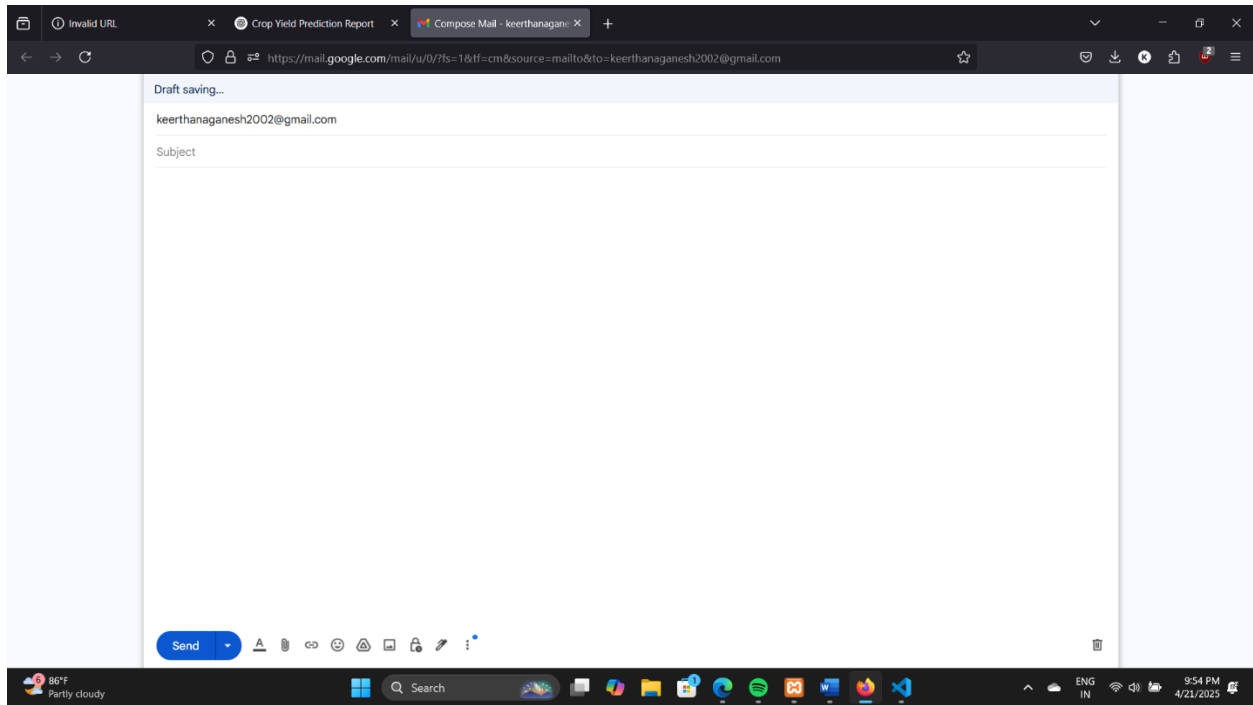
Screenshot 4: Login page



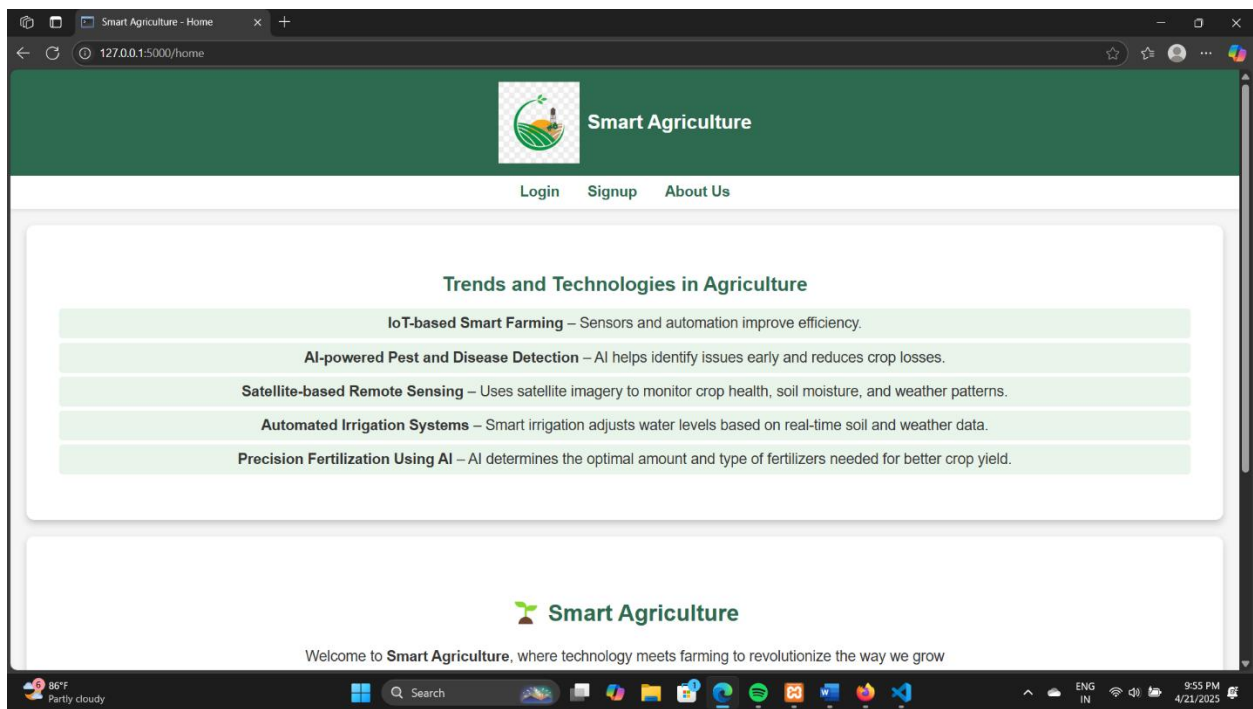
Screenshot 5: Signup page

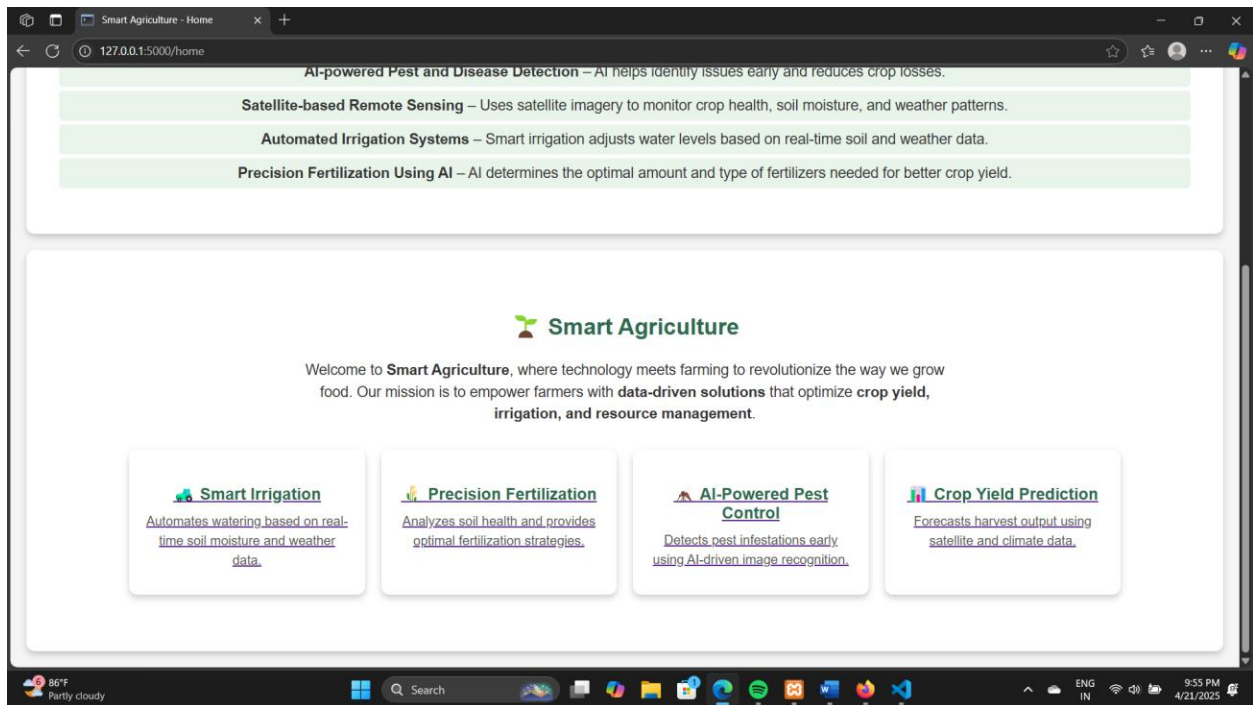


Screenshot 6: About us page

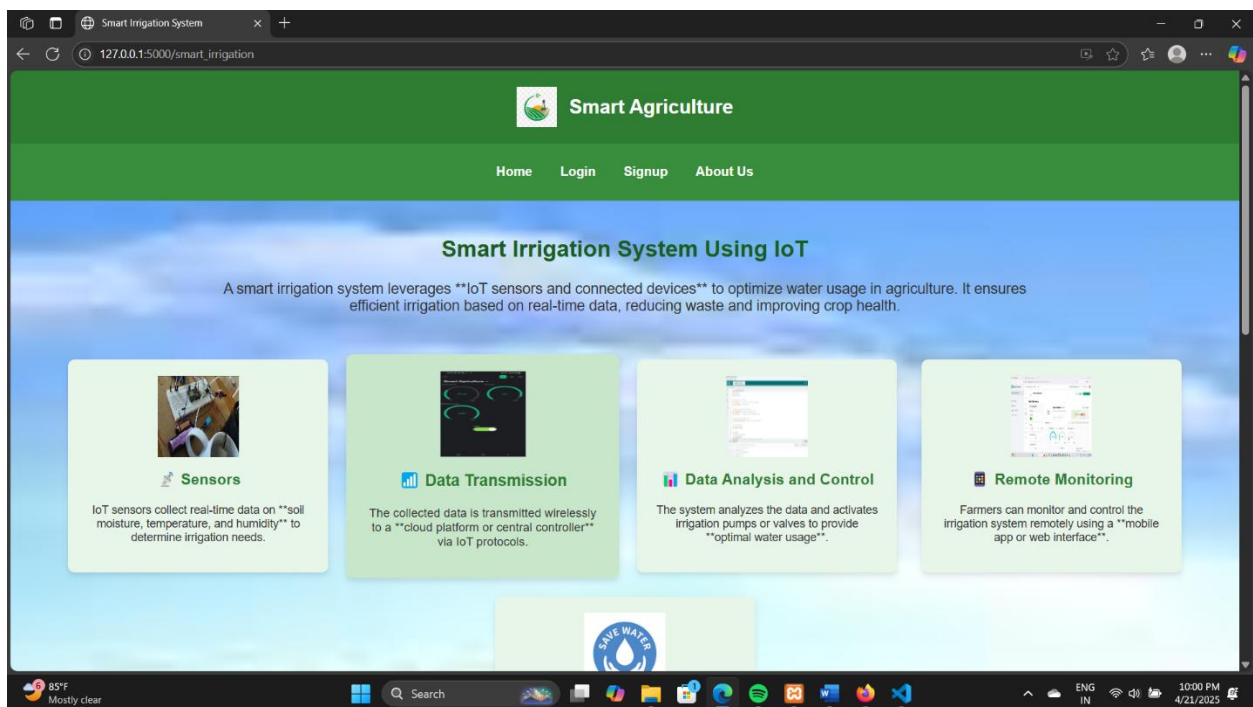


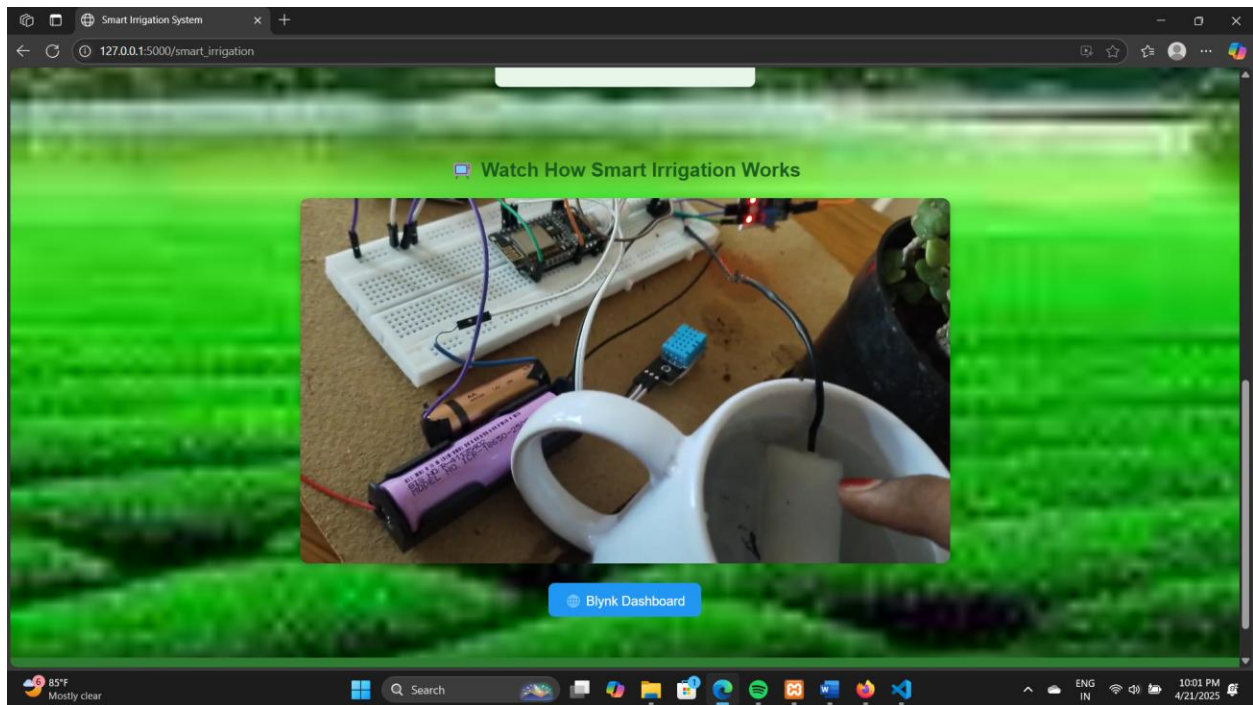
Screenshot 7: Email request to the developer



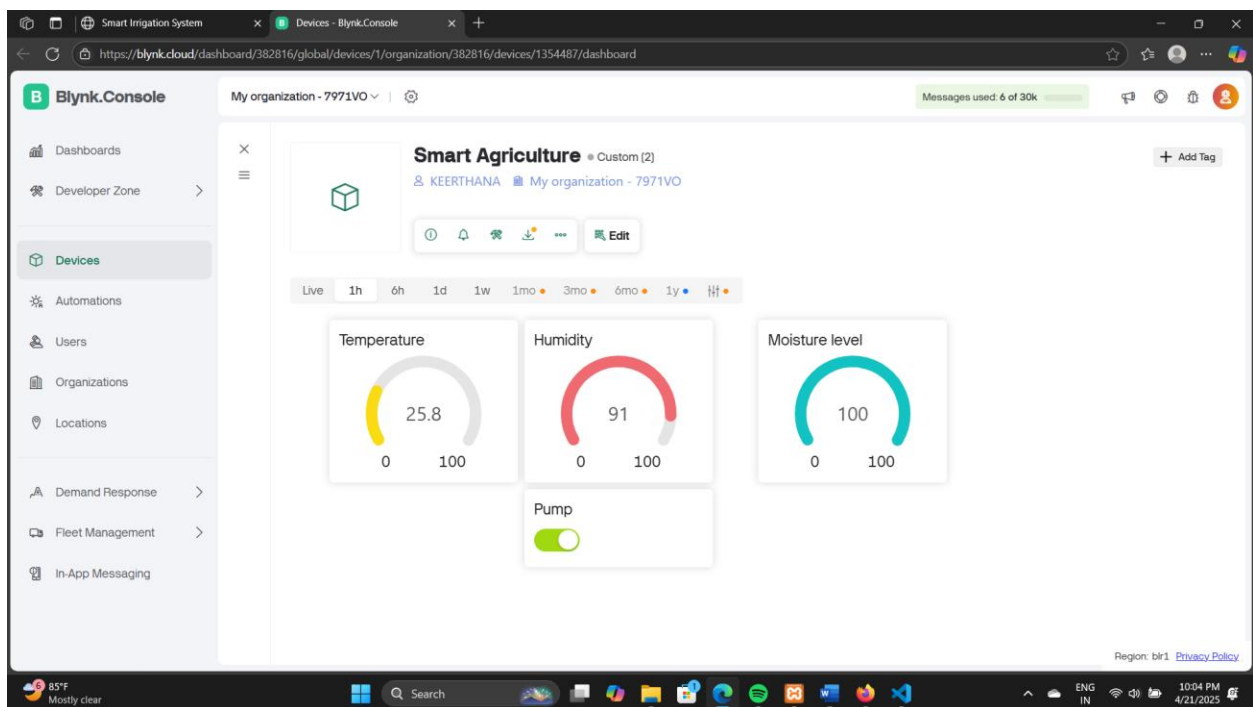


Screenshot 8: Home page

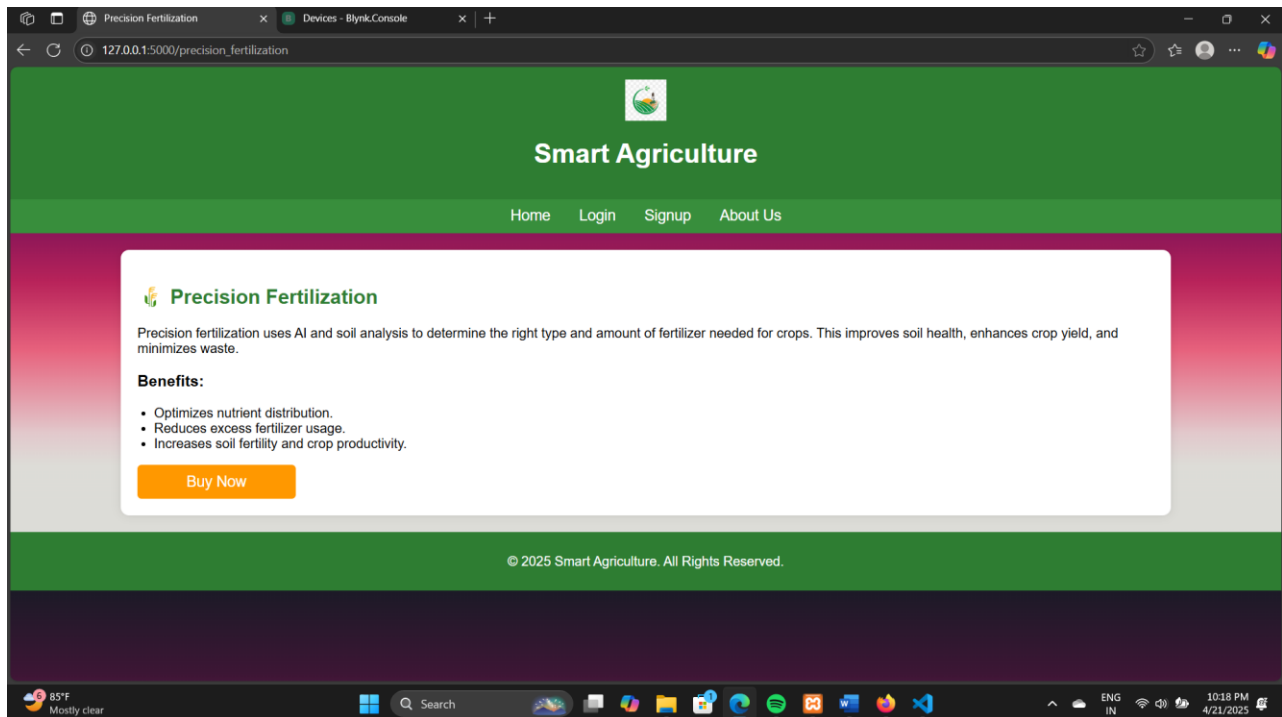




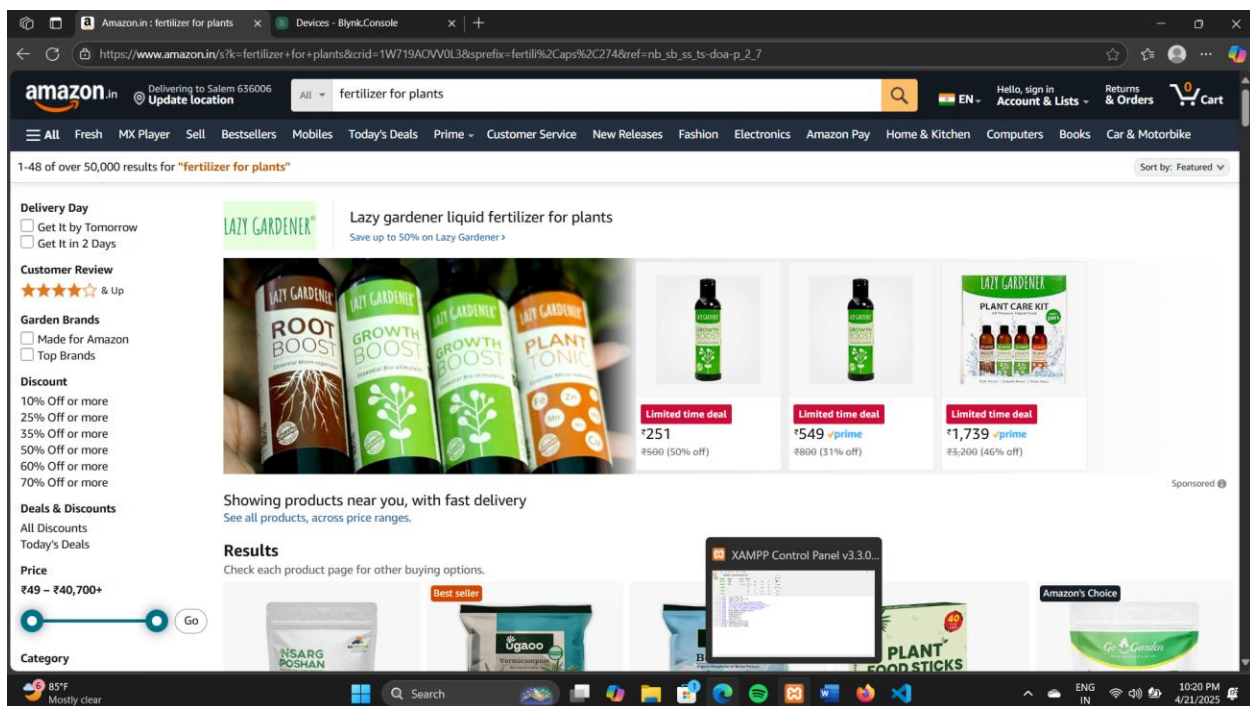
Screenshot 9: Smart irrigation using IoT



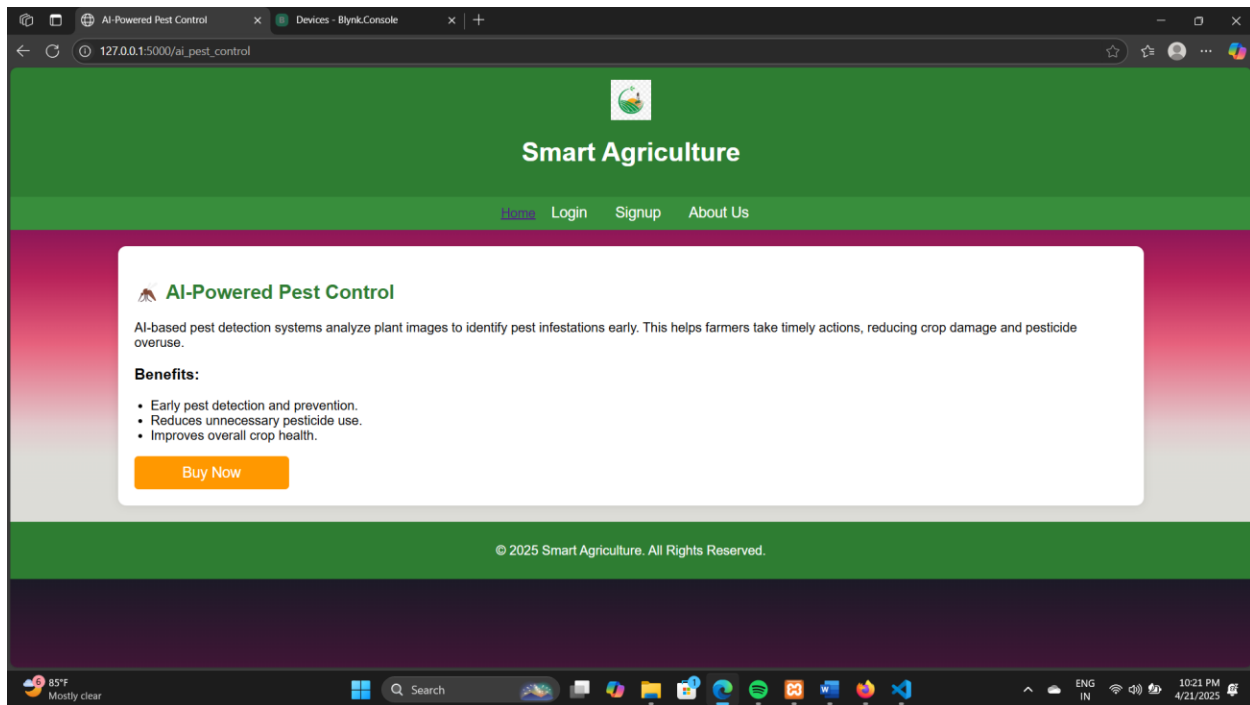
Screenshot 10: Blynk web dashboard



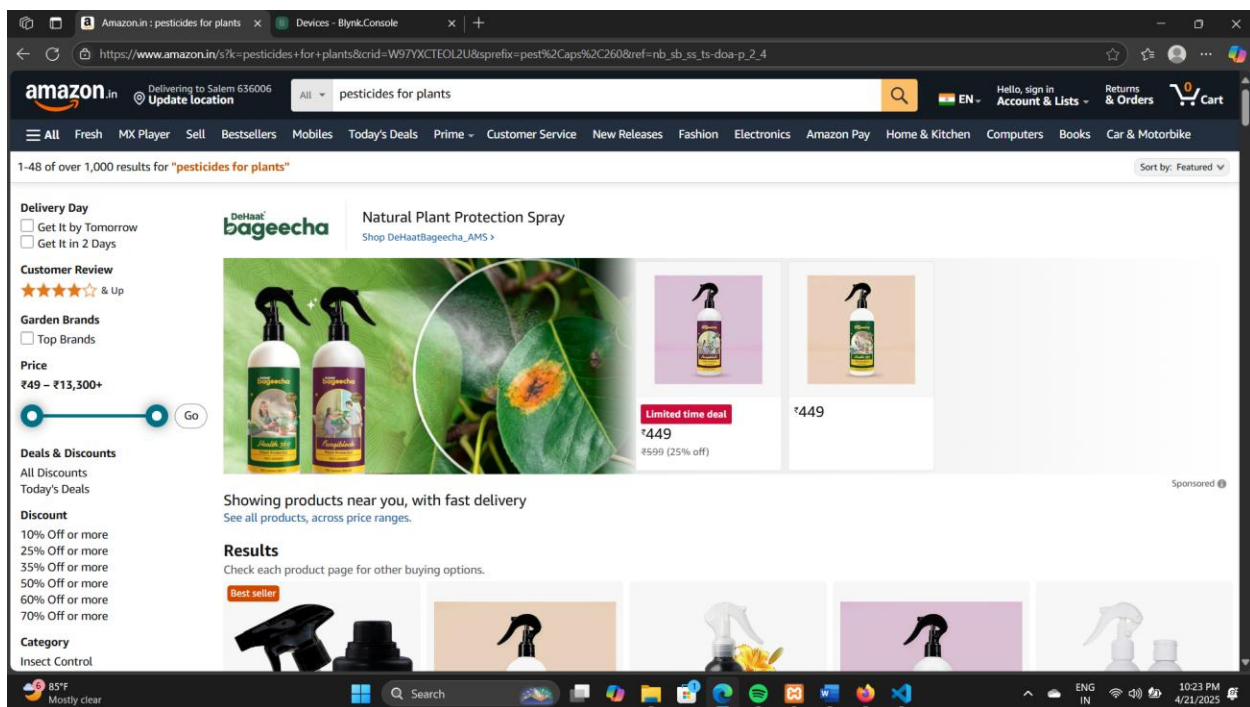
Screenshot 11: Precision fertilization page



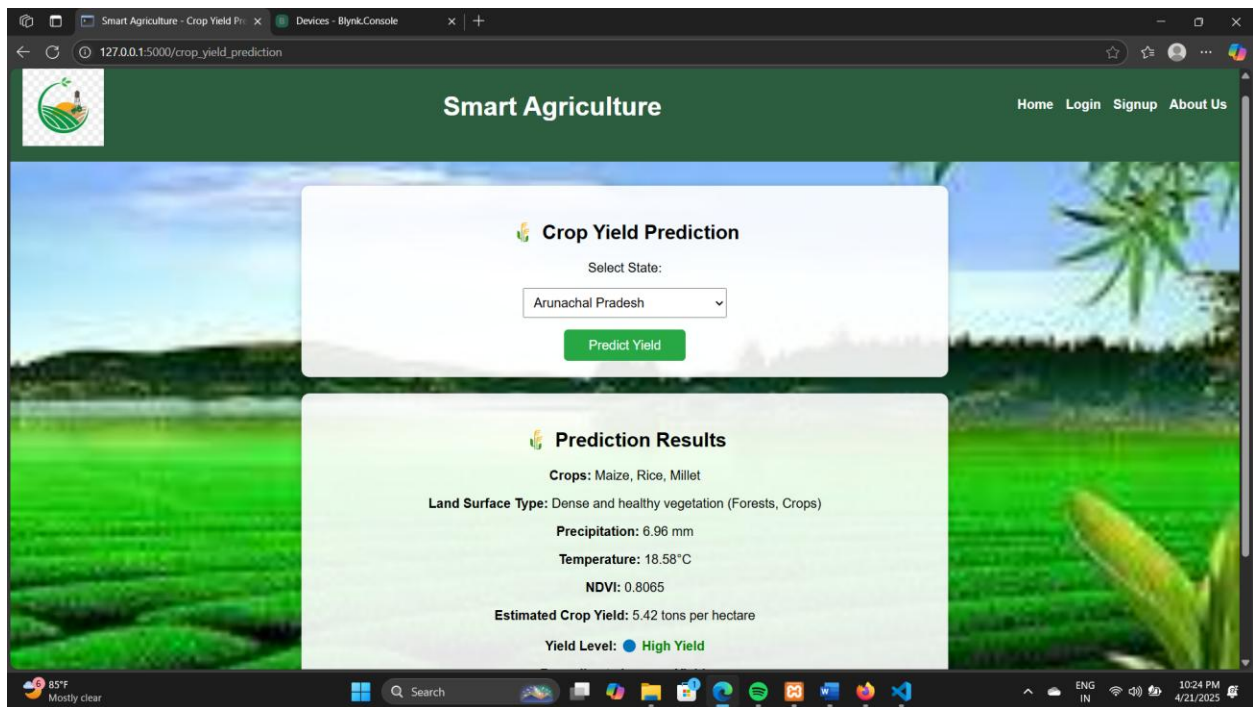
Screenshot 12: Purchase fertilizer



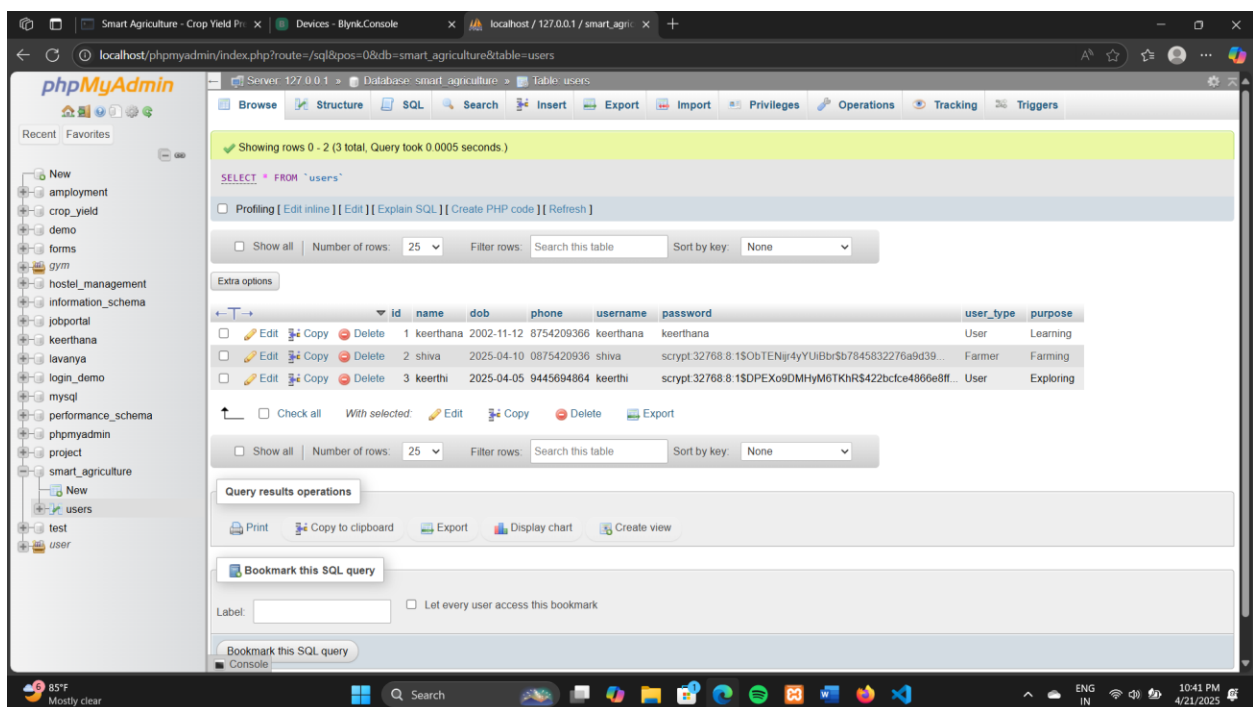
Screenshot 13: Pest control page



Screenshot 14: Purchase pest control



Screenshot 15: Crop yield prediction facilities.



Screenshot 16: Database connectivity.

References:

- [1] Liakos, K. G., Busato, P., Moshou, D., Pearson, S., & Bochtis, D. (2018). *Machine Learning in Agriculture: A Review*. *Sensors*, 18(8), 2674. <https://doi.org/10.3390/s18082674>
[ResearchGate+1PMC+1](#)
- [2] Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). *Deep learning in agriculture: A survey*. *Computers and Electronics in Agriculture*, 147, 70–90. <https://doi.org/10.1016/j.compag.2018.02.016>
[ResearchGate+5IRTA Pubpro+5ScienceDirect+5](#)
- [3] Arvind, K., Singh, R., & Sharma, P. (2021). *Role of AI/ML in Precision Farming: A Review*. *Journal of Precision Agriculture*, 12(3), 45–58. <https://doi.org/10.1016/j.jpa.2021.03.005>
- [4] Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). *Google Earth Engine: Planetary-scale geospatial analysis for everyone*. *Remote Sensing of Environment*, 202, 18–27. <https://doi.org/10.1016/j.rse.2017.06.031>
- [5] Jeong, J. H., Resop, J. P., Mueller, N. D., Fleisher, D. H., Yun, K., Butler, E. E., ... & Kim, S. H. (2016). *Random forests for global and regional crop yield predictions*. *PLOS ONE*, 11(6), e0156571. <https://doi.org/10.1371/journal.pone.0156571>
- [6] Becker-Reshef, I., Justice, C., Sullivan, M., Vermote, E., Tucker, C. J., Anyamba, A., ... & Schmaltz, J. (2010). *Monitoring Global Croplands with Coarse Resolution Earth Observations: The Global Agriculture Monitoring (GLAM) Project*. *Remote Sensing*, 2(6), 1589–1609. <https://doi.org/10.3390/rs2061589>
- [7] Lobell, D. B., & Burke, M. B. (2010). *On the use of statistical models to predict crop yield responses to climate change*. *Agricultural and Forest Meteorology*, 150(11), 1443–1452. <https://doi.org/10.1016/j.agrformet.2010.07.008>
- [8] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). *Using Deep Learning for Image-Based Plant Disease Detection*. *Frontiers in Plant Science*, 7, 1419. <https://doi.org/10.3389/fpls.2016.01419>

- [9] Chlingaryan, A., Sukkarieh, S., & Whelan, B. (2018). Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: A review. *Computers and Electronics in Agriculture*, 151, 61–69. <https://doi.org/10.1016/j.compag.2018.05.012>
- [10] You, J., Li, X., Low, M., Lobell, D., & Ermon, S. (2017). Deep Gaussian Process for Crop Yield Prediction Based on Remote Sensing Data. *Proceedings of the 31st AAAI Conference on Artificial Intelligence*. <https://ojs.aaai.org/index.php/AAAI/article/view/10739>
- [11] Pantazi, X. E., Moshou, D., & Tamouridou, A. A. (2016). Automated leaf disease detection in different crop species through image features analysis and One Class Classifiers. *Computers and Electronics in Agriculture*, 130, 113–123. <https://doi.org/10.1016/j.compag.2016.10.003>
- [12] Shanmugapriya, P., Balasubramani, K., & Thirumurugan, P. (2020). Machine Learning Techniques for Smart Agriculture: A Survey. *Journal of Ambient Intelligence and Humanized Computing*. <https://doi.org/10.1007/s12652-020-02168-2>
- [13] Drusch, M., Del Bello, U., Carlier, S., Colin, O., Fernandez, V., Gascon, F., ... & Bargellini, P. (2012). Sentinel-2: ESA's Optical High-Resolution Mission for GMES Operational Services. *Remote Sensing of Environment*, 120, 25–36. <https://doi.org/10.1016/j.rse.2011.11.026>
- [14] Wolfert, S., Ge, L., Verdouw, C., & Bogaardt, M.-J. (2017). Big Data in Smart Farming – A review. *Agricultural Systems*, 153, 69–80. <https://doi.org/10.1016/j.agsy.2017.01.023>
- [15] Kamir, E., Waldner, F., & Hochman, Z. (2020). Estimating wheat yields with remote sensing and minimal field data. *Scientific Reports*, 10, 12202. <https://doi.org/10.1038/s41598-020-68715-7>